UNIVERSITA' DEGLI STUDI DI NAPOLI FEDERICO II

Scuola Politecnica e delle Scienze di Base
Dipartimento di Ingegneria Elettrica e Tecnologie dell'Informazione

Corso di Laurea Magistrale in Ingegneria Informatica

# *SDN Project Work (Firewall) - Network and Cloud Infrastructures*

Anno Accademico 2024/2025

**Professori**
Prof. Giorgio Ventre
Prof. Roberto Canonico
Prof. Alessio Botta

**Candidati**
Matteo Cuomo. matr: M63001756

Antonio Tufo. matr: M63001727

Francesco Prisco. matr: M63001738

# Indice

**1 Introduzione e Background** **1**

1.1 Contesto . . . . . 1

1.2 Progetto di partenza . . . . . 2

1.3 Obiettivi di questo lavoro . . . . . 3

**2 Problemi riscontrati e Soluzioni introdotte** **6**

2.1 Lack of Modular Detection and Mitigation Design . . 8

2.2 Over-blocking . . . . . 10

2.2.1 Codice pre-patch . . . . . 10

2.2.2 Codice post-patch . . . . . 11

2.3 Controller-centric blocking decisions . . . . . 14

2.4 Detection Limited to Classic DoS Patterns . . . . . 16

2.5 Inflexible Blocking/Unblocking Policy . . . . . 19

2.5.1 Codice pre-patch . . . . . 19

2.5.2 Codice post-patch . . . . . 19

2.6 Soglia Dinamica Adattiva (EWMA) . . . . . 23

2.6.1 Codice pre-patch . . . . . 23

2.6.2 Codice post-patch . . . . . 24

2.6.3 Integrazione con il Policy Engine . . . . . 25

2.7 Progettazione della Topologia Estesa (Wide Topology) 27
2.7.1 Progettazione della nuova topologia estesa . . . 27
2.7.2 Integrazione con il controller . . . . . 29
2.7.3 Obiettivi di validazione futura . . . . . 30

**3 Feature Extra Aggiunte 33**
3.1 Rate Limiting con Meter (LIMIT) . . . . . 34
3.1.1 Idea di base . . . . . 35
3.1.2 Vantaggi . . . . . 35
3.1.3 Implementazione . . . . . 36
3.2 Blackhole temporaneo verso la vittima (BLACKHOLE) 37
3.2.1 Idea di base . . . . . 38
3.2.2 Vantaggi . . . . . 38
3.2.3 Implementazione . . . . . 39

**4 Testing e Validazione delle Modifiche 41**
4.1 Metodologia di Test . . . . . 41
4.2 Test: Over-Blocking . . . . . 42
4.2.1 Obiettivo . . . . . 42
4.2.2 Scenario di Test . . . . . 43
4.2.3 Comandi di Esecuzione . . . . . 44
4.2.4 Evidenze Sperimentali . . . . . 44
4.2.5 Log del Controller . . . . . 48
4.2.6 Risultati e Validazione . . . . . 50
4.3 Test: Burst Attack Detection . . . . . 50
4.3.1 Scenario di Test . . . . . 51
4.3.2 Comandi di esecuzione . . . . . 51

4.3.3 Evidenze sperimentali . . . . . 52
4.3.4 Risultati e validazione . . . . . 54
4.4 Test: Funzionalità di Blackhole . . . . . 55
4.4.1 Obiettivo . . . . . 55
4.4.2 Scenario di Test . . . . . 55
4.4.3 Comandi di esecuzione . . . . . 56
4.4.4 Evidenze sperimentali . . . . . 57
4.4.5 Risultati e validazione . . . . . 59
4.4.6 Criteri di superamento . . . . . 60
4.5 Test: Flexible Blocking / Unblocking Policy . . . . . 61
4.5.1 Obiettivo . . . . . 61
4.5.2 Test 1 – Automatic Unblock e Cooldown dinamico 62
4.5.3 Evidenze sperimentali . . . . . 63
4.5.4 Risultati e validazione . . . . . 67
4.5.5 Test 2 – Whitelist MAC (Host affidabili) . . . . 67
4.5.6 Evidenze sperimentali . . . . . 69
4.5.7 Risultati e validazione . . . . . 71
4.5.8 Test 3 – Blocklist REST API e blocco manuale 72
4.5.9 Evidenze sperimentali . . . . . 73
4.5.10 Risultati e validazione . . . . . 75

**5 Conclusioni 77**

# Capitolo 1

# Introduzione e Background

## 1.1 Contesto

Negli ultimi anni le architetture di rete basate su *Software Defined Networking* (SDN) hanno acquisito un ruolo centrale nella ricerca e nello sviluppo di soluzioni di sicurezza. L'approccio SDN separa il piano di controllo da quello di forwarding, permettendo di implementare logiche avanzate di monitoraggio e mitigazione degli attacchi direttamente a livello di controller. In particolare, l'utilizzo del framework *Ryu* consente di sviluppare applicazioni di sicurezza in grado di interagire in tempo reale con gli switch *OpenFlow*, istruendoli a bloccare o limitare traffico sospetto.

Uno degli scenari di attacco più rilevanti è rappresentato dai *Denial*

*of Service* (DoS) e *Distributed Denial of Service* (DDoS), nei quali un host malevolo sovraccarica un nodo bersaglio con un volume eccessivo di traffico, causando congestione e perdita di pacchetti per gli utenti legittimi. Il progetto di laboratorio, in questo contesto, ha come obiettivo la progettazione e l'implementazione di un firewall SDN in grado di:

- rilevare pattern anomali di traffico riconducibili ad attacchi DoS,
- attivare contromisure automatiche per salvaguardare i flussi legittimi.

## 1.2 Progetto di partenza

Il lavoro di quest'anno non parte da zero, ma si basa sul *Project Work* sviluppato dai colleghi nell'Anno Accademico 2023–2024. La traccia originale richiedeva di:

- implementare in Mininet una topologia con più host e switch,
- generare traffico normale (TCP) e malevolo (UDP ad alto bitrate) verso un server,
- misurare l'impatto dell'attacco sulle prestazioni dei flussi legittimi,
- introdurre un meccanismo di monitoraggio periodico delle statistiche di porta (`PortStatsRequest/Reply`),

- definire una soglia di throughput oltre la quale scatta l'allarme,
- installare regole di blocco sugli switch per mitigare l'attacco.

La soluzione dei colleghi ha soddisfatto questi obiettivi implementando:

- una **topologia** con tre host principali (H1 attaccante, H2 utente legittimo, H3 server) e quattro switch interconnessi,
- un sistema di **monitoraggio** multithread che raccoglie periodicamente le statistiche di porta,
- una **politica di remediation** basata sul blocco delle porte che superano una soglia di traffico definita,
- una gestione **dinamica dello sblocco**, che rimuove le regole quando il traffico torna sotto soglia,
- alcune **feature opzionali**, come la riduzione dell'impatto collaterale sugli host legittimi e l'integrazione di un bot Telegram per il monitoraggio.

## 1.3 Obiettivi di questo lavoro

Partendo dall'implementazione di riferimento dello scorso anno, l'obiettivo del presente progetto è stato quello di **evolvere il sistema**

**di difesa SDN contro attacchi DoS/DDoS** introducendo soluzioni efficaci alle principali limitazioni riscontrate, mantenendo al tempo stesso una struttura modulare, estendibile e facilmente validabile su Mininet.

In particolare, il lavoro ha perseguito i seguenti obiettivi principali:

- **Analizzare e superare i limiti residui** della precedente soluzione, in particolare l'*over-blocking*, la mancanza di modularità e l'uso di soglie statiche non adattive.

- **Progettare un'architettura completamente modulare**, basata su componenti indipendenti: `Monitor`, `PolicyEngine`, `Enforcer` e `DataStore`, coordinati dal controller `SDNDefender`.

- **Introdurre una logica di detection adattiva a livello di flusso** (flow-level detection), dotata di **soglie dinamiche** basate su *EWMA (Exponential Weighted Moving Average)* e meccanismi di rilevamento di traffico *bursty* o *stealth*.

- **Implementare strategie di mitigazione progressive**, con un approccio *limit* $\rightarrow$ *block*, che consente di limitare temporaneamente i flussi sospetti prima del blocco definitivo, riducendo così i falsi positivi.

- **Integrare una difesa DDoS aggregata** mediante la rilevazione di traffico distribuito verso la stessa destinazione e l'attiva-

zione di un **blackhole temporaneo** multi-switch per le vittime sovraccaricate.

- **Aggiungere un'interfaccia REST** per la gestione dinamica di *whitelist* e *blocklist*, permettendo l'interazione diretta dell'amministratore o di applicazioni esterne con il controller.

- **Validare il sistema** tramite una campagna di test su Mininet, utilizzando la topologia base composta da 4 host e 4 switch, con traffico UDP generato tramite `iperf3`.

Nel suo insieme, il progetto rappresenta un **miglioramento incrementale e funzionale** rispetto alla versione precedente, avvicinando il prototipo SDN Defender a un contesto realistico di difesa distribuita contro attacchi DoS e DDoS su reti Software Defined.

# Capitolo 2

# Problemi riscontrati e Soluzioni introdotte

Prima di analizzare nel dettaglio i singoli interventi, è utile fornire un quadro complessivo delle criticità individuate nel progetto originale e delle relative soluzioni implementate in questa nuova versione.
Il controller sviluppato lo scorso anno, sebbene funzionante, presentava alcuni limiti significativi:

- **Lack of Modular Detection and Mitigation Design**: il progetto era strutturato in modo monolitico, senza una chiara separazione tra monitoraggio, policy e enforcement, rendendo complessa l'estensione e la manutenzione del codice.

- **Over-blocking**: il traffico veniva bloccato a livello di porta, causando l'interruzione anche dei flussi legittimi che transitavano

dallo stesso switch o dalla stessa interfaccia.

- **Controller-centric decisions**: la logica di detection e di enforcement era concentrata unicamente nel controller, senza possibilità di interazione esterna o gestione dinamica delle policy (es. tramite API).

- **Detection Limited to Classic DoS Patterns**: il meccanismo di rilevamento si basava esclusivamente su soglie statiche di throughput, risultando inefficace contro attacchi *burst* o *stealth DoS* con traffico intermittente o non costante.

- **Inflexible Blocking/Unblocking Policy**: le regole di blocco venivano applicate e rimosse senza alcun meccanismo di cooldown temporale o gestione di whitelist e blocklist esterne, causando potenziali blocchi prolungati anche di utenti legittimi.

- **Static Threshold**: le soglie di rilevamento del traffico erano statiche e fissate a priori, senza adattarsi alle variazioni dinamiche della rete. Questo comportava falsi positivi in presenza di picchi momentanei e falsi negativi in scenari di traffico crescente. *La nuova versione introduce una soglia dinamica basata su media mobile esponenziale (EWMA)*, capace di adattarsi in tempo reale al comportamento dei flussi monitorati.

- **Topology Sensitivity**: la versione precedente era fortemente legata a una topologia specifica, limitando la generalizzabilità

dei risultati. Nel nuovo progetto è stata sviluppata una **topologia estesa (topology_wide.py)** composta da più switch e host distribuiti, al fine di validare l'efficacia del sistema anche in scenari più complessi e realistici. (La validazione sperimentale su tale topologia sarà oggetto di test futuri.)

Nel corso di questo progetto, ciascuno di questi problemi è stato affrontato e risolto con un approccio incrementale. Nei paragrafi seguenti verranno mostrati, per ogni criticità, il codice originale (*pre-patch*), la soluzione adottata (*post-patch*) e un breve commento sulle differenze.

## 2.1 Lack of Modular Detection and Mitigation Design

Un primo problema rilevante della soluzione originale era la totale assenza di modularità. Tutte le componenti — monitoraggio del traffico, rilevamento di anomalie e applicazione delle regole — erano concentrate in un unico file del controller. Questa struttura monolitica rendeva il codice complesso da manutenere, difficile da estendere e poco flessibile in caso di evoluzioni future del progetto.

Per superare questo limite, il nuovo controller è stato riorganizzato secondo un'architettura a moduli indipendenti, ciascuno con una responsabilità ben definita:

- **monitor.py**: raccoglie periodicamente le statistiche di porte e flussi, convertendole in valori di throughput e pacchetti per secondo, senza introdurre logiche di soglia.

- **policy.py**: implementa la logica decisionale, valutando i dati forniti dal monitor. Qui si trovano le regole di detection classiche basate su soglia, i meccanismi di **soglia dinamica adattiva (EWMA)**, la *burst/stealth detection* e la gestione di scenari di DDoS distribuito.

- **enforcement.py**: applica le decisioni prese dalla policy sugli switch tramite flow rules OpenFlow. Supporta blocchi di porta, blocchi di flusso, rate limiting con meter e blackhole temporanei.

- **datastore.py**: mantiene lo stato condiviso in maniera thread-safe, conservando serie temporali di porte e flussi, lo stato degli allarmi e le whitelist/blocklist esterne.

- **config.py**: raccoglie tutti i parametri configurabili in un unico punto, rendendo più semplice l'adattamento del controller a scenari differenti senza modificare il codice.

- **models.py**: definisce le strutture dati fondamentali (chiavi di porta e di flusso, azioni possibili, delta statistici), garantendo coerenza e riusabilità in tutto il progetto.

- **controller_app.py**: funge da orchestratore, istanziando i vari moduli e gestendo gli eventi di Ryu, mantenendo un ruolo leggero e pulito rispetto al progetto precedente.

Grazie a questa ristrutturazione, il progetto è diventato molto più leggibile e scalabile. L'aggiunta di nuove tecniche di rilevamento o di strategie di mitigazione non richiede più di modificare un unico blocco di codice complesso, ma può essere ottenuta estendendo i moduli esistenti o aggiungendone di nuovi in maniera coerente con l'architettura.

## 2.2 Over-blocking

Uno dei principali limiti della soluzione dello scorso anno era la gestione del traffico anomalo a livello di *porta*. Quando una porta superava la soglia di throughput, il controller installava una regola di *drop* che bloccava indiscriminatamente tutto il traffico in ingresso da quella porta. Questo approccio causava il cosiddetto **over-blocking**: non solo il flusso malevolo veniva interrotto, ma anche eventuali flussi legittimi che condividevano la stessa porta.

### 2.2.1 Codice pre-patch

Nella Figura 2.2.1 è riportata la funzione `lock_flow` della versione precedente del controller: il match era basato unicamente sul campo `in_port`, e ciò comportava il blocco totale della porta.

```
def lock_flow (self, ev, port_no):

    ofproto = ev.msg.datapath.ofproto
    parser = ev.msg.datapath.ofproto_parser

    match = parser.OFPMatch(in_port=port_no)

    instructions=[]

    flow_mod = parser.OFPFlowMod(datapath=ev.msg.datapath, priority=2, match=match, instructions=instructions,
                                 command=ofproto.OFPFC_ADD, out_port= ofproto.OFPP_ANY,
                                   out_group = ofproto.OFPG_ANY, flags=ofproto.OFPFF_SEND_FLOW_REM)

    ev.msg.datapath.send_msg(flow_mod)
    print(RED + "Blocked traffic on port %s of switch %s " + RESET, port_no, ev.msg.datapath.id)
```

Figura 2.1: `controller.py` pre-patch

### 2.2.2 Codice post-patch

Per risolvere il problema dell'over-blocking, nel nuovo progetto è stata introdotta una mitigazione *granulare* a livello di flusso. La logica è stata ristrutturata nel modulo `policy.py`, che valuta le statistiche dei flussi e decide se limitarne la banda o bloccarli del tutto, e nel modulo `enforcement.py`, che applica effettivamente le regole sugli switch.

```
scope = self.cfg.get("detect_scope", "port")
if scope == "flow":
    for key in self.ds.get_all_flow_keys():
        self._evaluate_flow(key)
else:
    for key in self.ds.get_all_port_keys():
        self._evaluate_port(key)

# Analisi DDoS aggregato solo se attivata e in modalità flow
if self.cfg.get("ddos_enable", True) and scope == "flow":
    self._detect_ddos_aggregate()
```

Figura 2.2: Modifica `policy.py` (1)

```python
if over and v < 3 * threshold:
    now = time.time()
    if not hasattr(self, "_last_limit_ts"):
        self._last_limit_ts = {}
    last_time = self._last_limit_ts.get(sig, 0)
    cooldown = 8
    if now - last_time < cooldown:
        self._limit_counter[sig] = self._limit_counter.get(sig, 0) + 1
        if self._limit_counter[sig] >= 5:
            self.enforcer.enqueue(Action(ActionType.BLOCK, enforce_key))
            app.logger.info(
                f"{Color.RED}[BLOCKED]{Color.RESET} after {self._limit_counter[sig]} consecutive LIMITs {enforce_key}"
            )
            self._limit_counter[sig] = 0
        return

    app.logger.info(
        f"{Color.YELLOW}LIMIT FLOW {Color.RESET}{enforce_key} ({reason}) "
        f"count={self._limit_counter[sig]} bps={int(v)} thr={int(threshold)} "
        f"cv={feats['cv']:.2f} peak/ewma={(feats['maxv']/(feats['ewma'] or 1)):.2f} (ingress={ingress})"
    )

    self.enforcer.enqueue(Action(
        kind=ActionType.LIMIT,
        key=enforce_key,
        params={"rate_kbps": int(self.cfg.get('limit_rate_kbps', 1000))}
    ))
```

Figura 2.3: Modifica `policy.py` (2)

```python
def _block_flow(self, key: FlowKey) -> None:
    """Installa una regola di DROP per un flusso specifico."""
    dp = self.app.datapaths.get(key.dpid)
    if not dp or key in self._blocked_flows:
        return
    ofp = dp.ofproto
    parser = dp.ofproto_parser
    match = self._build_flow_match(dp, key)
    mod = parser.OFPFlowMod(
        datapath=dp,
        priority=self.cfg["drop_priority"],
        match=match,
        instructions=[],  # DROP
        command=ofp.OFPFC_ADD,
        out_port=ofp.OFPP_ANY,
        out_group=ofp.OFPG_ANY,
        idle_timeout=0,
        hard_timeout=0,
    )
    dp.send_msg(mod)
    self._blocked_flows.add(key)
    self.app.logger.info(f"Installing DROP (flow) {key}")
```

Figura 2.4: Modifica `enforcement.py`

Rispetto al design originale, la nuova versione introduce una serie di miglioramenti significativi:

- Le decisioni di rilevamento vengono ora prese sulla base di **soglie dinamiche adattive (EWMA)**, che si aggiornano nel tempo

in funzione dell'andamento del traffico reale, evitando blocchi causati da picchi momentanei.

- È stato introdotto un meccanismo di **rate limiting progressivo**: i flussi sospetti vengono inizialmente limitati a 1 Mbps per alcuni cicli di osservazione (fino a 5 tentativi consecutivi). Solo in assenza di miglioramento viene applicato il blocco definitivo.
- Le azioni di mitigazione (LIMIT, BLOCK, UNBLOCK) vengono eseguite in modo **asincrono e modulare** dal componente `Enforcer`, separato dal controller principale, garantendo maggiore robustezza e coerenza tra i datapath.

Grazie a questa ristrutturazione, il problema dell'over-blocking è stato eliminato. Il controller ora distingue tra:

- **Blocchi di porta**, ancora possibili ma solo in casi estremi.
- **Blocchi di flusso**, mirati sul traffico anomalo.
- **Rate limiting**, che riduce la banda dei flussi sopra soglia ma non critici a 1 Mbps, preservando la connettività.

In questo modo il traffico legittimo non viene più interrotto a causa di un attacco DoS proveniente da un singolo host collegato alla stessa porta, e il sistema è in grado di reagire in maniera selettiva, adattiva e progressiva a seconda della gravità dell'anomalia rilevata.

## 2.3 Controller-centric blocking decisions

Nella versione originale del progetto, la logica di rilevamento e quella di enforcement erano fortemente intrecciate: il controller stesso, nel momento in cui rilevava una condizione di traffico anomalo, applicava immediatamente le regole di blocco sugli switch. Questo approccio **controller-centric** presentava due principali svantaggi:

- riduceva la chiarezza del codice, poiché detection e enforcement erano implementati nello stesso flusso logico,
- rendeva più complesso gestire politiche avanzate, come il rate limiting o il blackhole temporaneo, che richiedono una fase di decisione separata dall'applicazione.

Per risolvere questa limitazione, il nuovo progetto ha introdotto una chiara **separazione dei ruoli**:

- il modulo `policy.py` si occupa unicamente di prendere decisioni, valutando le statistiche raccolte e producendo azioni sotto forma di oggetti di tipo `Action`.
- il modulo `enforcement.py` riceve queste azioni e le traduce in regole OpenFlow concrete (FlowMod), applicandole in modo idempotente sugli switch.

La comunicazione tra i due moduli avviene tramite una **coda di azioni** gestita dall'`Enforcer`. Tale coda viene svuotata in ordine FIFO

dalla funzione interna `_drain()`, che esegue le operazioni in modo asincrono e thread-safe, distinguendo tra azioni di tipo `BLOCK`, `UNBLOCK`, `LIMIT` e `BLACKHOLE`. Questo meccanismo disaccoppia detection ed enforcement, garantendo coerenza e idempotenza anche in presenza di più datapath attivi.

Grazie a questa architettura, il controller diventa molto più leggibile e manutenibile:

1. ogni modulo si occupa di una responsabilità specifica (decisione o applicazione),
2. le politiche di mitigazione possono essere facilmente estese o modificate senza toccare il codice di basso livello che interagisce con gli switch.

In sintesi, la logica decisionale non è più "hardcoded" nel controller stesso, ma è stata delegata a un motore di policy dedicato, che emette comandi standardizzati. L'`Enforcer` li applica poi sugli switch in modo sicuro e coerente, rendendo il sistema più modulare, estendibile e vicino a un design industriale.

```
self.enforcer.enqueue(Action(
    kind=ActionType.LIMIT,
    key=enforce_key,
    params={"rate_kbps": int(self.cfg.get('limit_rate_kbps', 1000))}
))
```

Figura 2.5: `policy.py` – generazione dell'azione `LIMIT`

```python
def _drain(self) -> None:
    """Esegue le azioni accodate, distinguendo tra porta e flusso."""
    while self._queue:
        a = self._queue.popleft()

        # --- Azione su FLUSSO ---
        if isinstance(a.key, FlowKey):
            if a.kind == ActionType.BLOCK:
                self._block_flow(a.key)
            elif a.kind == ActionType.UNBLOCK:
                self._unblock_flow(a.key)
            elif a.kind == ActionType.LIMIT:
                rate = a.params.get("rate_kbps", 1000) if a.params else 1000
                self._limit_flow(a.key, rate)
```

Figura 2.6: `enforcement.py` – esecuzione della coda di azioni (_drain)

## 2.4 Detection Limited to Classic DoS Patterns

Il progetto originale utilizzava un meccanismo di rilevamento molto semplice, basato esclusivamente sul confronto del throughput di una porta con una soglia fissa. Se il traffico superava tale valore, il sistema scattava in allarme e attivava immediatamente la fase di mitigazione. Questo approccio funzionava per attacchi DoS "classici", caratterizzati da un volume costante e molto elevato di traffico, ma risultava inefficace in scenari più subdoli:

- attacchi **burst**, in cui il traffico malevolo si presenta sotto forma di picchi intermittenti molto intensi,
- attacchi **stealth DoS**, caratterizzati da flussi al di sotto della soglia media, ma con elevata variabilità nel tempo.

Per affrontare questa limitazione, nel nuovo progetto è stato introdotto un insieme di metriche aggiuntive all'interno del modulo `policy.py`. Oltre alla semplice soglia di throughput, ora il sistema calcola:

- la **varianza** e la **deviazione standard** dei campioni di traffico raccolti in una finestra temporale mobile,
- il **coefficiente di variazione** (*coefficient of variation*), che misura il rapporto tra deviazione standard e media del flusso,
- il rapporto **picco/EWMA**, che confronta il valore massimo osservato con la media esponenziale mobile del traffico, utile a individuare spike anomali anche sotto la soglia assoluta,
- un'**analisi della variabilità temporale**, che consente di distinguere flussi costanti da flussi oscillanti o intermittenti.

Grazie a queste misure, la rilevazione non si basa più su una soglia statica, ma su una **baseline dinamica adattiva**, aggiornata in tempo reale tramite una media mobile esponenziale (EWMA). In questo modo il controller è in grado di riconoscere anche attacchi non costanti, distinguendo tra normali fluttuazioni di traffico e picchi anomali.
Quando tali condizioni vengono rilevate, la `PolicyEngine` sceglie dinamicamente la strategia di mitigazione più appropriata: applicare un **rate limiting progressivo** ai flussi sospetti o procedere al blocco definitivo in caso di persistenza.

Il sistema di detection risulta quindi più **robusto e flessibile**, adattandosi in tempo reale anche a scenari di attacchi moderni e più sofisticati.

```
def _stats_and_burst(self, samples, alpha: float = 0.2):
    if not samples:
        return dict(n=0, mean=0.0, std=0.0, ewma=0.0, maxv=0.0, cv=0.0)

    vals = [float(s.bps) for s in samples]
    n = len(vals)
    m = 0.0
    m2 = 0.0
    k = 0
    ewma = None
    maxv = 0.0

    for v in vals:
        k += 1
        delta = v - m
        m += delta / k
        m2 += delta * (v - m)
        ewma = v if ewma is None else (alpha * v + (1.0 - alpha) * ewma)
        if v > maxv:
            maxv = v

    mean = m
    var = (m2 / (k - 1)) if k > 1 else 0.0
    std = math.sqrt(var)
    cv = (std / mean) if mean > 0 else 0.0
    return dict(n=n, mean=mean, std=std, ewma=ewma, maxv=maxv, cv=cv)
```

Figura 2.7: Funzione _stats_and_burst in `policy.py`

```
# 4) Rilevamento di burst/stealth
burst_window = float(self.cfg.get("burst_window_s", 6.0))
samples = self._recent_flow_samples(key, burst_window)
feats = self._stats_and_burst(samples, alpha=float(self.cfg.get("ewma_alpha", 0.2)))

cv_min = float(self.cfg.get("burst_cv_min", 0.8))
peak_ratio = float(self.cfg.get("burst_peak_ratio", 2.5))
micro = float(self.cfg.get("micro_threshold_bps", 200_000))
min_mean_for_burst = max(micro, float(self.cfg.get("min_mean_for_burst", 500_000)))

bursty = False
if feats["mean"] >= min_mean_for_burst:
    cv_ok = feats["cv"] >= cv_min
    peak_ok = (feats["ewma"] > 0 and (feats["maxv"] / feats["ewma"]) >= peak_ratio)
    if cv_ok and peak_ok:
        bursty = True
```

Figura 2.8: Combinazione tra soglia adattiva e burst detection all'interno della funzione _evaluate_flow in `policy.py`

## 2.5 Inflexible Blocking/Unblocking Policy

Nel progetto originale, la politica di blocco e sblocco era molto rigida: quando un flusso superava la soglia di traffico, il controller installava immediatamente una regola di drop; al contrario, quando il traffico scendeva sotto la soglia, la regola veniva rimossa subito. Questa gestione portava a due problemi principali:

- i flussi legittimi potevano rimanere bloccati più a lungo del necessario, oppure venivano sbloccati troppo presto,
- un attaccante poteva facilmente riprendere l'attacco subito dopo lo sblocco, forzando un continuo ciclo di blocco/sblocco.

### 2.5.1 Codice pre-patch

Nella versione originale lo sblocco era legato solo al ritorno sotto soglia: appena il traffico diminuiva, il controller chiamava la funzione di `unlock`, senza alcuna ulteriore condizione.

### 2.5.2 Codice post-patch

Nella nuova versione sono stati introdotti tre miglioramenti principali.

**1. Cooldown temporale.** Nel modulo `policy.py` (Figura 2.9) ogni blocco viene associato a un timestamp, salvato in una strut-

tura interna (`_last_block_ts`). Prima di emettere un'azione di `UNBLOCK`, la policy calcola il tempo trascorso dall'ultimo blocco:

- se l'intervallo è minore del valore configurato (`unblock_cooldown_s` in `config.py`), lo sblocco non viene applicato,
- se l'intervallo è maggiore, il flusso viene sbloccato e i contatori vengono azzerati.

Questo meccanismo introduce una gestione **stateful**, basata sul tempo e sullo storico delle azioni, evitando oscillazioni troppo rapide e rendendo la policy più stabile.

```
# Dopo il periodo di cooldown, UNBLOCK automatico
elif alarm_on:
    last_block = self._last_block_ts.get(self._flow_sig(key), 0)
    age = time.time() - last_block
    if age >= self.cfg.get("unblock_cooldown_s", 15):
        self.enforcer.enqueue(Action(ActionType.UNBLOCK, key))
        alarm_on = False
        counter = 0
```

Figura 2.9: Policy Engine: gestione dello sblocco con cooldown minimo.

**2. Whitelist di MAC.** Nel modulo `datastore.py` (Figura 2.10) è stata aggiunta la possibilità di definire una **whitelist di indirizzi MAC**. Gli host presenti in questa lista non possono mai essere bloccati, anche se superano la soglia. Questo consente di proteggere dispositivi considerati "critici" o attendibili (ad esempio server di management o gateway interni). La whitelist può essere gestita anche

dinamicamente tramite le **API REST** esposte dal controller, senza dover modificare il codice.

```
def add_whitelist_mac(self, mac: str) -> None:
    with self._lock:
        if mac:
            self._ext_whitelist_macs.add(mac.lower())

def remove_whitelist_mac(self, mac: str) -> None:
    with self._lock:
        if mac:
            self._ext_whitelist_macs.discard(mac.lower())

def list_whitelist_macs(self) -> List[str]:
    with self._lock:
        return sorted(self._ext_whitelist_macs)

def is_mac_whitelisted(self, mac: str) -> bool:
    with self._lock:
        return mac.lower() in self._ext_whitelist_macs if mac else False
```

Figura 2.10: DataStore: funzioni di gestione della whitelist MAC.

**3. Blocklist esterne.** Sempre in `datastore.py` (Figura 2.11) sono state introdotte anche le **blocklist esterne**, ovvero insiemi di flussi che possono essere forzati in DROP dall'amministratore o da moduli esterni tramite API REST. In questo caso il controller non si basa su metriche di traffico, ma applica immediatamente il blocco imposto dall'esterno, mantenendo comunque il tracciamento dello stato nel `DataStore`.

```python
def add_block_flow(self, fk: FlowKey) -> None:
    with self._lock:
        self._ext_block_flows.add(fk)

def remove_block_flow(self, fk: FlowKey) -> None:
    with self._lock:
        self._ext_block_flows.discard(fk)

def list_block_flows(self) -> Iterable[FlowKey]:
    with self._lock:
        return list(self._ext_block_flows)

def external_block_flows(self) -> Set[FlowKey]:
    with self._lock:
        return set(self._ext_block_flows)
```

Figura 2.11: DataStore: funzioni di gestione della blocklist esterna.

Grazie a questi cambiamenti, la politica di blocco e sblocco è passata da un modello rigido e poco realistico a un approccio molto più **flessibile e adattivo**. L'introduzione di un cooldown evita continui cicli di blocco/sblocco, le whitelist garantiscono che host affidabili non vengano mai interrotti, mentre le blocklist permettono di estendere la protezione con input esterni. Inoltre, l'integrazione con le API REST rende il sistema **interattivo e gestibile in tempo reale**. In prospettiva, la struttura modulare del controller consente di estendere facilmente queste policy anche a criteri più complessi, come priorità QoS o regole basate su livello IP o trasporto, senza modificare il nucleo logico del sistema.

## 2.6 Soglia Dinamica Adattiva (EWMA)

Nel progetto originale, la soglia di rilevamento del traffico (`threshold_bps`) era definita come un valore fisso, valido per tutte le porte e tutti i flussi. Questo approccio si rivelava efficace solo in condizioni di rete stabili, ma mostrava forti limiti nei casi in cui il carico variava nel tempo:

- si generavano **falsi positivi** quando un flusso legittimo superava temporaneamente la soglia fissa;
- si verificavano **falsi negativi** quando un attacco restava costantemente al di sotto della soglia statica;
- il sistema risultava fortemente **dipendente dalla topologia** e dalle condizioni di traffico con cui era stato tarato.

In sintesi, la detection basata su una soglia statica non era in grado di adattarsi alla naturale variabilità del traffico reale.

### 2.6.1 Codice pre-patch

Nel codice della versione precedente, il controller confrontava semplicemente il throughput di una porta o di un flusso con il valore `threshold_bps`. Non esisteva alcuna forma di adattamento dinamico: la decisione di bloccare veniva presa ogni volta che il valore istantaneo superava la soglia.

### 2.6.2 Codice post-patch

Per risolvere questa limitazione, il nuovo sistema ha introdotto un meccanismo di **soglia dinamica adattiva** basato su una *media mobile esponenziale* (EWMA – Exponential Weighted Moving Average). Questo meccanismo aggiorna costantemente la soglia di rilevamento in base all'andamento reale del traffico, permettendo al controller di adattarsi in modo automatico e continuo.
La logica è implementata nel modulo `datastore.py` tramite due funzioni:

- `update_dynamic_threshold()` per la modalità *port-level*,
- `update_dynamic_flow_threshold()` per la modalità *flow-level*.

Ciascuna di esse calcola una media esponenziale del bitrate osservato, secondo la regola:

$$T_{nuovo} = \alpha \cdot v + (1 - \alpha) \cdot T_{precedente}$$

dove $\alpha$ è un coefficiente di adattamento (0.3 per le porte, 0.1 per i flussi) e $v$ rappresenta il valore di traffico misurato nel ciclo corrente.
Per garantire la stabilità del sistema:

- vengono scartati valori negativi dovuti a reset o overflow,

- la soglia dinamica viene mantenuta tra 0 e 3× la soglia statica, così da evitare oscillazioni o derive anomale.

```python
def update_dynamic_flow_threshold(self, key: FlowKey, value: float, alpha: float = 0.1):
    """
    Aggiorna la soglia dinamica per flusso con un filtro EWMA più "lento"
    (alpha ridotto) e applica limiti di sicurezza:
    - scarta valori negativi (reset contatori o overflow)
    - mantiene la soglia tra 0 e 3x la soglia statica
    """
    with self._lock:
        prev = self._dynamic_flow_thresholds[key]

        # 🔹 Evita input negativi dovuti a reset contatori
        if value < 0:
            value = 0.0

        # 🔹 Calcolo EWMA
        ewma = value if prev == 0 else alpha * value + (1 - alpha) * prev

        # 🔹 Recupera soglia statica come riferimento
        try:
            base_thr = float(self.cfg.get("threshold_bps", 2_000_000))
        except AttributeError:
            base_thr = 2_000_000

        # 🔹 Impone limiti di sicurezza
        max_allowed = 3 * base_thr
        ewma = max(0.0, min(ewma, max_allowed))

        self._dynamic_flow_thresholds[key] = ewma

def get_dynamic_flow_threshold(self, key: FlowKey) -> float:
    with self._lock:
        return self._dynamic_flow_thresholds.get(key, 0.0)
```

Figura 2.12: `DataStore`: funzione `update_dynamic_flow_threshold()`. Il valore EWMA viene aggiornato ad ogni campione, mantenendolo entro limiti di sicurezza.

### 2.6.3 Integrazione con il Policy Engine

Il modulo `policy.py` utilizza queste soglie dinamiche per ogni flusso monitorato. Ad ogni intervallo di polling:

- viene aggiornato il valore EWMA tramite `update_dynamic_flow_thre`

- la soglia dinamica $T_d$ viene calcolata come 1.2× EWMA,
- la soglia effettiva utilizzata per il confronto è $\max(T_d, T_{static})$.

In questo modo, il sistema mantiene un limite minimo di sensibilità (dato dalla soglia statica) ma adatta dinamicamente il comportamento alle variazioni del traffico in tempo reale.

```
# 3) Soglia dinamica adattiva (EWMA)
self.ds.update_dynamic_flow_threshold(key, v)
dyn_thr = 1.2 * self.ds.get_dynamic_flow_threshold(key)
threshold = max(dyn_thr, float(self.cfg["threshold_bps"]))
over = v > threshold
```

Figura 2.13: `PolicyEngine`: utilizzo della soglia dinamica nella funzione `_evaluate_flow()`. Il controller recupera l'EWMA dal DataStore, lo moltiplica per 1.2 e lo confronta con la soglia statica prima di decidere se applicare un'azione di LIMIT o BLOCK.

L'introduzione della soglia dinamica adattiva ha portato numerosi vantaggi:

- **Adattamento automatico** alle fluttuazioni del traffico in tempo reale.
- **Riduzione dei falsi positivi**, poiché flussi legittimi ad alta intensità non vengono più bloccati erroneamente.
- **Maggiore robustezza** e stabilità del sistema in topologie o carichi diversi.

Questo miglioramento risolve pienamente la criticità della **"Static Threshold"** evidenziata nel documento originale del progetto, rendendo

il controller SDN Defender adatto a scenari di rete dinamici e a lungo termine.

## 2.7 Progettazione della Topologia Estesa (Wide Topology)

La versione iniziale del progetto SDN Defender era basata su una topologia semplice, composta da quattro switch e quattro host. Tutti i nodi si trovavano nello stesso dominio di rete, connessi allo stesso livello di accesso, e il traffico malevolo proveniva sempre da un unico punto di ingresso. Questa configurazione, sebbene utile nelle prime fasi di sviluppo, presentava alcuni limiti significativi:

- non permetteva di verificare il comportamento del controller su più switch o in scenari di forwarding multi-hop;
- non consentiva di simulare attacchi distribuiti (multi-source DDoS);
- le politiche di mitigazione e soglia risultavano calibrate su una rete troppo semplice, priva di dinamiche realistiche di congestione o ritardo.

### 2.7.1 Progettazione della nuova topologia estesa

Per superare questi limiti, è stata progettata una nuova topologia multilivello, descritta nel file `topology_wide.py`, ispirata alle architet-

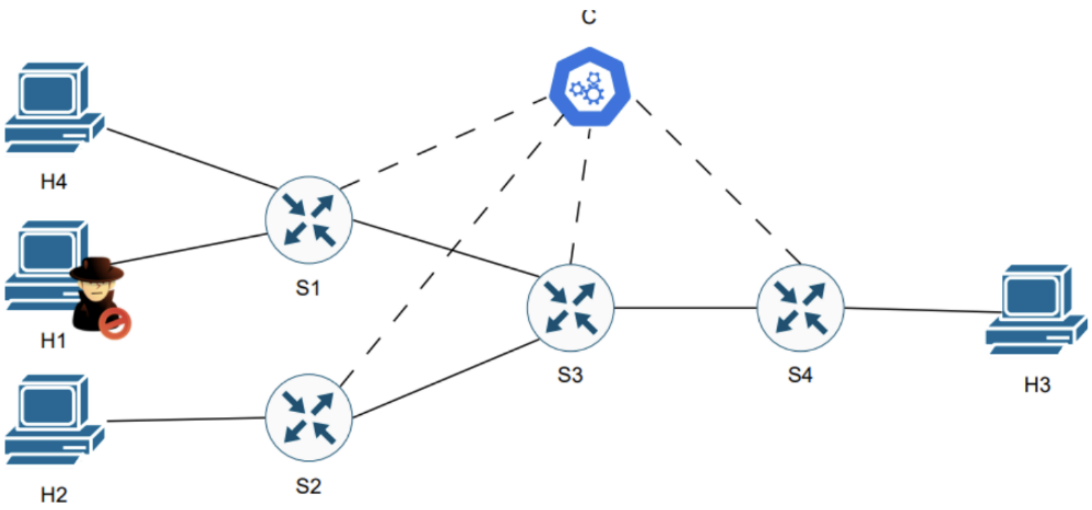


Figura 2.14: Topologia originale a 4 switch e 4 host. Tutti gli host sono collegati agli stessi edge switch, limitando la possibilità di simulare scenari distribuiti.

ture reali di rete aziendale. La topologia segue un modello gerarchico a tre livelli:

- **Core layer:** uno switch ad alta capacità che funge da dorsale centrale.
- **Aggregation layer:** tre switch intermedi che collegano il core ai domini edge.
- **Edge layer:** cinque switch di accesso che ospitano gli host terminali.
- **Host:** sei nodi totali (`h1-h6`), di cui tre attaccanti (`h1, h3, h5`) e tre legittimi (`h2, h4, h6`).

Gli host attaccanti sono distribuiti su edge switch differenti per poter generare attacchi multi-sorgente e verificare, in futuro, la capacità del controller di isolare i flussi malevoli solo presso gli ingressi effettivi, senza penalizzare il traffico legittimo.

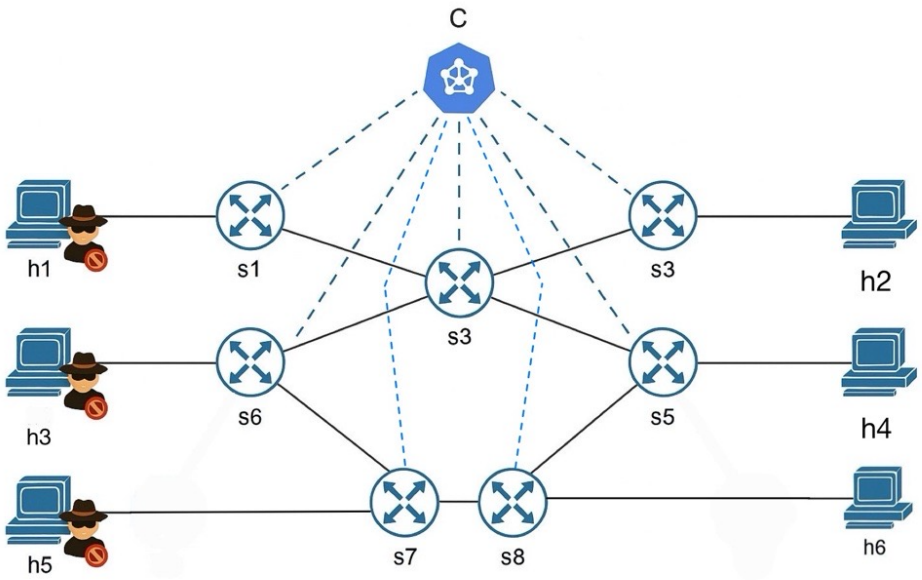


Figura 2.15: Topologia estesa progettata (9 switch, 6 host). Il design segue una struttura a tre livelli *core–aggregation–edge*, consentendo di simulare attacchi distribuiti provenienti da più domini di accesso.

### 2.7.2 Integrazione con il controller

La topologia estesa è già integrata nel progetto tramite lo stesso controller modulare `SDNDefender`, eseguito come `RemoteController` sull'indirizzo predefinito `127.0.0.1:6653`. Il file `topology_wide.py` include inoltre una funzione helper `start_network()` che consente di avviare automaticamente:

- la rete Mininet con i nove switch e sei host;
- i server `iperf3` sui nodi vittima;
- i flussi UDP di attacco generati dagli host malevoli.

Questo setup riproduce scenari più realistici, con ritardi inter-switch, collegamenti multipli e aggregazione del traffico da diversi ingressi.

### 2.7.3 Obiettivi di validazione futura

La nuova topologia non è stata ancora sottoposta a test sperimentali, ma è stata progettata per supportare le seguenti verifiche:

- **Scalabilità del controller:** valutare il comportamento del motore di detection e enforcement in presenza di più datapath attivi.
- **Mitigazione distribuita:** verificare che le azioni di `BLOCK`, `LIMIT` e `BLACKHOLE` vengano applicate solo sugli ingressi effettivi dei flussi malevoli.
- **Stabilità delle soglie adattive:** analizzare l'evoluzione delle soglie EWMA in ambienti con topologie più profonde e ritardi variabili.
- **Test DDoS aggregato:** validare l'attivazione del meccanismo di `BLACKHOLE` in presenza di traffico distribuito proveniente da più domini.

Attraverso questa espansione, il progetto SDN Defender evolve da un semplice prototipo su topologia piatta a una piattaforma più **scalabile e realistica**, in grado di affrontare scenari di rete complessi. La campagna di test sperimentali su tale topologia costituisce un obiettivo immediato per le prossime fasi di sviluppo.

| **Problema** | **Soluzione Introdotta** |
|---|---|
| Lack of Modular Detection and Mitigation Design | L'architettura è stata riorganizzata in moduli indipendenti: `monitor.py`, `policy.py`, `enforcement.py`, `datastore.py`, `config.py`, `models.py`, e un `controller_app.py` più leggero e orchestratore. |
| Over-blocking | Il blocco avviene ora a livello di singolo flusso (`FlowKey`) e non più solo a livello di porta. È stato inoltre introdotto il **rate limiting progressivo** per i flussi borderline, riducendo i falsi positivi e preservando il traffico legittimo. |
| Controller-centric blocking decisions | La logica è stata separata: `policy.py` produce le decisioni, mentre `enforcement.py` le applica tramite regole OpenFlow gestite da una **coda di azioni asincrona**, garantendo modularità e idempotenza. |
| Detection limited to classic DoS patterns | La detection è stata estesa con analisi di **burst** e metriche statistiche (varianza, coefficiente di variazione, rapporto picco/EWMA, analisi della variabilità temporale), rendendo il sistema più robusto contro attacchi *stealth* o non costanti. |
| Inflexible blocking unblocking policy | Sono stati introdotti meccanismi di **cooldown temporale** per lo sblocco, una **whitelist MAC** per host affidabili e **blocklist esterne** gestibili via API REST, che consentono blocchi forzati da amministratori o applicazioni esterne. |
| Static Threshold | È stata implementata una **soglia dinamica adattiva** basata su filtro EWMA, che si aggiorna automaticamente in base al traffico legittimo. La soglia effettiva è calcolata come massimo tra quella statica e la dinamica, evitando falsi blocchi e migliorando la stabilità complessiva. |
| Topology Sensitivity | È stata **progettata e integrata una topologia wide** con più switch e host (3 attaccanti e 3 legittimi, 9 switch totali), pensata per validare il comportamento del controller in scenari complessi e multi-switch. La validazione sperimentale non è stata prevista. |

Tabella 2.1: Riepilogo dei problemi iniziali e delle soluzioni adottate nel progetto SDN Defender.

# Capitolo 3

# Feature Extra Aggiunte

In questo capitolo vengono illustrate le **funzionalità aggiuntive** implementate all'interno del sistema di difesa SDN Defender, progettate per rendere le azioni di mitigazione più flessibili, proporzionali e intelligenti rispetto alla gravità del traffico rilevato.
L'obiettivo principale di queste feature è fornire una **risposta adattiva** agli eventi: non sempre un flusso anomalo deve essere completamente bloccato. In molti casi è preferibile limitarne temporaneamente la banda, monitorando l'evoluzione del comportamento, mentre in scenari estremi può essere necessario adottare una misura drastica come il blocco totale del traffico verso una determinata vittima. Le funzionalità introdotte e analizzate in questo capitolo sono:

- **Rate Limiting con Meter (LIMIT)**: applicazione di una limitazione di banda (1 Mbps di default) per i flussi che superano la soglia dinamica ma non sono considerati critici. Questo

meccanismo è gestito dal modulo `PolicyEngine`, che invia un'azione `LIMIT` all'`Enforcer`, il quale installa una regola di meter OpenFlow dedicata. Se il flusso rimane anomalo per più cicli consecutivi, il rate limiting evolve automaticamente in un `BLOCK` definitivo.

- **Blackhole temporaneo verso la vittima (BLACKHOLE)**: attivato in presenza di un attacco DDoS distribuito, quando il traffico complessivo verso una stessa destinazione supera la soglia aggregata configurata. In questo caso, il controller impone un **blackhole multi-switch** per una durata prefissata (20 s di default), bloccando completamente il traffico verso la vittima e registrando l'evento nel sistema di logging.

Queste funzionalità estendono ulteriormente la logica di difesa del controller, rendendo il sistema più **reattivo e resiliente** anche in scenari di attacco distribuito o di traffico irregolare. Nei paragrafi seguenti verranno analizzati nel dettaglio il funzionamento dei due meccanismi, la loro integrazione con la pipeline del controller e i relativi effetti sul comportamento complessivo del sistema.

## 3.1 Rate Limiting con Meter (LIMIT)

Una prima estensione implementata riguarda l'utilizzo delle *Meter Table* di OpenFlow per applicare un meccanismo di **rate limiting** sui

flussi sospetti, prima di arrivare a un blocco definitivo.

### 3.1.1 Idea di base

Quando un flusso supera la **soglia effettiva** (statica vs dinamica EWMA) ma non raggiunge livelli considerati critici, invece del blocco totale viene applicata una **limitazione di banda** — pari a `1 Mbps` di default (parametro `limit_rate_kbps` in `config.py`). In questo modo il traffico viene contenuto senza interrompere completamente la comunicazione, e la policy può osservare se il comportamento rientra entro valori normali.

### 3.1.2 Vantaggi

L'approccio LIMIT presenta diversi vantaggi:

- evita falsi positivi invasivi (non si blocca subito un flusso borderline);
- riduce l'impatto dell'attacco preservando la connettività applicativa;
- abilita una **reazione progressiva**: se il flusso resta sopra soglia per più cicli consecutivi, si *escalation* a `BLOCK` (logica LIMIT → BLOCK).

### 3.1.3 Implementazione

Dal punto di vista tecnico, il LIMIT avviene in due passaggi coordinati `PolicyEngine` → `Enforcer`:

1. la `PolicyEngine`, durante `_evaluate_flow()`, invece di emettere subito un `BLOCK` crea un'azione `ActionType.LIMIT` con il parametro `rate_kbps` (default 1000);

2. l'`Enforcer` riceve l'azione dalla coda, emette un `OFPMeterMod` con `meter_id = 1` e installa una `FlowMod` che associa **solo quel flusso** al meter, con priorità di drop configurata.

Il primo passaggio è visibile in Figura 3.1: all'interno del metodo `_drain()` viene intercettata l'azione `ActionType.LIMIT` e chiamata la funzione `_limit_flow()`, passando il rate configurato.

```
elif a.kind == ActionType.LIMIT:
    rate = a.params.get("rate_kbps", 1000) if a.params else 1000
    self._limit_flow(a.key, rate)
```

Figura 3.1: Gestione dell'azione `LIMIT` nel metodo `_drain()` dell'Enforcer.

Il secondo passaggio è mostrato in Figura 3.2: la funzione `_limit_flow()` crea/aggiorna un meter a `1000 kbps` (`OFPM_KBPS` con banda `Drop`) e installa una `FlowMod` che **matcha il *FlowKey*** (quindi niente over-blocking) e invia il traffico verso l'`OFPP_NORMAL`, applicando l'istruzione di meter.

```python
def _limit_flow(self, key: FlowKey, rate_kbps: int) -> None:
    """Installa un meter per limitare la banda di un flusso."""
    dp = self.app.datapaths.get(key.dpid)
    if not dp:
        return
    ofp = dp.ofproto
    parser = dp.ofproto_parser
    match = self._build_flow_match(dp, key)

    # crea/aggiorna meter id=1 (semplice demo)
    meter_mod = parser.OFPMeterMod(
        datapath=dp,
        command=ofp.OFPMC_ADD,
        flags=ofp.OFPMF_KBPS,
        meter_id=1,
        bands=[parser.OFPMeterBandDrop(rate=rate_kbps)]
    )
    dp.send_msg(meter_mod)

    inst = [
        parser.OFPInstructionMeter(meter_id=1),
        parser.OFPInstructionActions(
            ofp.OFPIT_APPLY_ACTIONS,
            [parser.OFPActionOutput(ofp.OFPP_NORMAL)]
        )
    ]
    mod = parser.OFPFlowMod(
        datapath=dp,
        priority=self.cfg["drop_priority"],
        match=match,
        instructions=inst,
        command=ofp.OFPFC_ADD,
        out_port=ofp.OFPP_ANY,
        out_group=ofp.OFPG_ANY,
        idle_timeout=0,
        hard_timeout=0,
    )
    dp.send_msg(mod)
```

Figura 3.2: Funzione `_limit_flow`: creazione del Meter e applicazione della limitazione di banda.

## 3.2 Blackhole temporaneo verso la vittima (BLACKHOLE)

Il secondo meccanismo di difesa introdotto nel sistema è il **blackhole temporaneo**, utilizzato in scenari di *DDoS massivo*. Quando il traffico aggregato verso una stessa destinazione supera di molto la soglia tollerabile, limitare la banda non è più sufficiente: è necessario

bloccare completamente ogni comunicazione verso la vittima per un intervallo di tempo predefinito.

### 3.2.1 Idea di base

L'azione di *blackhole* consiste nell'installazione di una regola di **DROP** su tutti i pacchetti destinati a un certo indirizzo IP o MAC. La regola viene propagata su tutti gli switch gestiti dal controller e rimane attiva per una durata di $N$ secondi (configurabile tramite il parametro `blackhole_duration_s` in `config.py`). Trascorso tale periodo, la regola viene rimossa automaticamente grazie al campo `hard_timeout`, evitando di isolare permanentemente la destinazione.

### 3.2.2 Vantaggi

Questo meccanismo garantisce:

- **protezione immediata** della rete e dei servizi in caso di DDoS critico;
- **semplicità di attuazione**, in quanto una sola regola di drop blocca tutto il traffico verso la vittima;
- **reversibilità automatica**, grazie alla rimozione programmata dopo $N$ secondi.

### 3.2.3 Implementazione

Il blackhole temporaneo è composto da due fasi principali:

1. **Decisione** (`policy.py`): durante la rilevazione di un attacco DDoS distribuito, il `PolicyEngine` verifica che il traffico aggregato verso una stessa destinazione superi la soglia complessiva configurata. Se `blackhole_enable=True`, viene emessa un'azione di tipo `ActionType.BLACKHOLE` con i parametri della vittima (MAC/IP) e la durata dell'isolamento.

2. **Enforcement** (`enforcement.py`): l'`Enforcer` riceve l'azione e invoca il metodo `_blackhole_dest()`, che installa su **tutti i datapath attivi** una regola di **DROP** con priorità elevata e `hard_timeout` impostato alla durata desiderata.

La logica di decisione è illustrata in Figura 3.3: il `PolicyEngine` rileva un picco di traffico anomalo verso una stessa destinazione e accoda l'azione di tipo `BLACKHOLE` per bloccare temporaneamente ogni flusso verso tale host.

```
if self.cfg.get("blackhole_enable", True):
    # Blackhole temporaneo verso la destinazione (MAC)
    duration_s = int(self.cfg.get("blackhole_duration_s", 20))
    self.enforcer.app.logger.info(
        f"[DDoS] BLACKHOLE dst={dst} total_bps={int(total)} flows={len(vals)} for {duration_s}s"
    )
    # Usiamo key fittizia; i parametri portano la destinazione
    self.enforcer.enqueue(Action(
        kind=ActionType.BLACKHOLE,
        key=PortKey(dpid=0, port_no=0),
        params={"victim_mac": dst, "victim_ip": None, "duration_s": duration_s}
    ))
```

Figura 3.3: Decisione del `PolicyEngine`: attivazione di un'azione `BLACKHOLE` in caso di DDoS aggregato verso una destinazione.

L'esecuzione effettiva dell'azione è mostrata in Figura 3.4. La funzione `_blackhole_dest()` costruisce un `FlowMod` con istruzioni vuote (`instructions=[]`), che produce un drop totale dei pacchetti destinati alla vittima. L'opzione `OFPFF_SEND_FLOW_REM` e il campo `hard_timeout` garantiscono la rimozione automatica della regola allo scadere del periodo configurato.

```python
def _blackhole_dest(self, victim_mac: Optional[str], victim_ip: Optional[str], duration_s: int) -> None:
    """
    Installa su TUTTI i datapath una regola di DROP che matcha la destinazione (MAC o IP).
    Usa hard_timeout=duration_s per rimuoversi automaticamente.
    """
    if not victim_mac and not victim_ip:
        self.app.logger.warning("BLACKHOLE requested without victim_mac or victim_ip")
        return

    for dpid, dp in self.app.datapaths.items():
        ofp = dp.ofproto
        parser = dp.ofproto_parser

        match_kwargs = {}
        if victim_ip:
            match_kwargs["eth_type"] = 0x0800
            match_kwargs["ipv4_dst"] = victim_ip
        elif victim_mac:
            match_kwargs["eth_dst"] = victim_mac

        match = parser.OFPMatch(**match_kwargs)

        mod = parser.OFPFlowMod(
            datapath=dp,
            priority=self.cfg["drop_priority"] + 5,
            match=match,
            instructions=[],       # DROP
            command=ofp.OFPFC_ADD,
            out_port=ofp.OFPP_ANY,
            out_group=ofp.OFPG_ANY,
            idle_timeout=0,
            hard_timeout=int(duration_s),
            flags=ofp.OFPFF_SEND_FLOW_REM,
        )
        dp.send_msg(mod)
        self.app.logger.info(
            f"Installing BLACKHOLE on dpid={dpid} dst="
            f"{victim_ip if victim_ip else victim_mac} "
            f"for {int(duration_s)}s"
        )
```

Figura 3.4: Funzione `_blackhole_dest()` dell'Enforcer: installazione multi-switch della regola di DROP e rimozione automatica dopo la durata configurata.

# Capitolo 4

# Testing e Validazione delle Modifiche

Dopo l'implementazione delle funzionalità e delle correzioni descritte nei capitoli precedenti, si è resa necessaria una fase di **testing** e **validazione** al fine di verificare l'efficacia delle soluzioni proposte. L'obiettivo di questo capitolo è quello di dimostrare, attraverso prove pratiche eseguite in ambiente *Mininet* e *Ryu Controller*, che ciascun problema individuato è stato correttamente risolto.

## 4.1 Metodologia di Test

Per validare le modifiche sono stati utilizzati i seguenti strumenti e procedure:

- **Topologia di rete**: la stessa topologia impiegata nella fase di

progettazione, con host (H1, H2, H3, H4) e switch opportunamente configurati.

- **Generazione del traffico**: utilizzo di comandi `iperf3` per simulare attacchi *UDP flood* e *TCP connections*, e comandi `ping` per monitorare la connettività tra host.
- **Log di sistema**: output del controller Ryu per osservare in tempo reale l'applicazione delle regole (`limit`, `block`, `unblock`) e la gestione delle soglie dinamiche.
- **Criteri di validazione**: assenza di *over-blocking*, applicazione corretta del rate limiting, mantenimento della connettività per traffico legittimo e rimozione dinamica delle regole quando l'attacco termina.

Nei paragrafi successivi verranno presentati i test relativi ad ogni problema risolto, mostrando il comportamento del sistema *prima* e *dopo* la correzione, con evidenze tratte direttamente dall'output del terminale.

## 4.2 Test: Over-Blocking

### 4.2.1 Obiettivo

Questo test ha lo scopo di verificare la risoluzione del problema di *Overblocking*, ossia la tendenza del controller, nelle versioni precedenti del sistema, ad applicare azioni di mitigazione eccessive bloccando

intere porte o interi switch d'ingresso, con conseguente impatto anche su flussi legittimi. L'obiettivo è dimostrare che, grazie all'introduzione del nuovo meccanismo di enforcement **flow-level**, il controller agisce ora in modo selettivo, isolando esclusivamente il flusso anomalo e preservando la continuità del traffico regolare.

### 4.2.2 Scenario di Test

- **Topologia:** composta da tre host (`H1`, `H2`, `H3`) connessi a uno switch OpenFlow gestito dal controller Ryu.
- **Ruoli:** `H1` agisce come host attaccante, `H2` genera traffico legittimo, mentre `H3` funge da server di destinazione.
- **Traffico generato:** due flussi UDP verso lo stesso server:
  - **H1 → H3:5201** – traffico malevolo ad alta intensità (50 Mbit/s).
  - **H2 → H3:5201** – traffico legittimo e stabile (1 Mbit/s).

Il controller monitora entrambi i flussi in tempo reale e applica misure di limitazione (`LIMIT`) o blocco (`BLOCK`) esclusivamente al traffico proveniente da `H1`, lasciando invariato quello proveniente da `H2`.

### 4.2.3 Comandi di Esecuzione

Il test è stato condotto aprendo tre terminali separati con `xterm`, uno per ciascun host della topologia. Di seguito i comandi utilizzati e la relativa descrizione:

```
# Su H3 (server)
iperf3 -s -p 5201
```

Avvia il server `iperf3` in ascolto sulla porta 5201.

```
# Su H1 (attaccante)
iperf3 -c 10.0.0.3 -p 5201 -u -b 50M -t 20 -i 1
```

Genera un flusso **UDP ad alta intensità** a 50 Mbit/s per 20 secondi verso la porta 5201 del server, simulando un attacco di saturazione (*DoS-like*).

```
# Su H2 (flusso legittimo)
iperf3 -c 10.0.0.3 -p 5201 -u -b 1M -t 20 -i 1
```

Genera un flusso **UDP costante e legittimo** a 1 Mbit/s per 20 secondi verso lo stesso server e la stessa porta.

### 4.2.4 Evidenze Sperimentali

Le Figure 4.1–4.4 mostrano le evidenze raccolte durante l'esperimento:

- **H3 (Server):** riceve due flussi simultanei; si osserva che il traffico generato da `H1` viene limitato e successivamente bloccato, mentre quello proveniente da `H2` rimane stabile.

- **H1 (Attaccante):** genera il traffico malevolo a 50 Mbit/s verso la porta 5201.

- **H2 (Legittimo):** mantiene un flusso costante a 1 Mbit/s non influenzato dalla mitigazione.

```
"Node: h3"
root@smag01:/home/smag01/vmshare/sdn_defender/topology# iperf3 -s -p 5201 &
[1] 6768
root@smag01:/home/smag01/vmshare/sdn_defender/topology# ------------------------
-----------------------------------
Server listening on 5201
-----------------------------------------------------------
Accepted connection from 10.0.0.1, port 48222
[  7] local 10.0.0.3 port 5201 connected to 10.0.0.1 port 56184
[ ID] Interval           Transfer     Bitrate         Jitter    Lost/Total Datag
rams
[  7]   0.00-1.00   sec   612 KBytes  5.01 Mbits/sec  1.743 ms  0/433 (0%)
[  7]   1.00-2.00   sec   713 KBytes  5.84 Mbits/sec  1.774 ms  0/504 (0%)
[  7]   2.00-3.00   sec   711 KBytes  5.83 Mbits/sec  0.974 ms  827/1330 (62%)
[  7]   3.00-4.00   sec   711 KBytes  5.82 Mbits/sec  0.985 ms  2090/2593 (81%)

[  7]   4.00-5.00   sec   594 KBytes  4.87 Mbits/sec  9.810 ms  5171/5591 (92%)

[  7]   5.00-6.00   sec  0.00 Bytes  0.00 bits/sec  9.810 ms  0/0 (0%)
[  7]   6.00-7.00   sec  0.00 Bytes  0.00 bits/sec  9.810 ms  0/0 (0%)
[  7]   7.00-8.00   sec  0.00 Bytes  0.00 bits/sec  9.810 ms  0/0 (0%)
[  7]   8.00-9.00   sec  0.00 Bytes  0.00 bits/sec  9.810 ms  0/0 (0%)
[  7]   9.00-10.00  sec  0.00 Bytes  0.00 bits/sec  9.810 ms  0/0 (0%)
[  7]  10.00-11.00  sec  0.00 Bytes  0.00 bits/sec  9.810 ms  0/0 (0%)
[  7]  11.00-12.00  sec  0.00 Bytes  0.00 bits/sec  9.810 ms  0/0 (0%)
[  7]  12.00-13.00  sec  0.00 Bytes  0.00 bits/sec  9.810 ms  0/0 (0%)
[  7]  13.00-14.00  sec  0.00 Bytes  0.00 bits/sec  9.810 ms  0/0 (0%)
[  7]  14.00-15.00  sec  0.00 Bytes  0.00 bits/sec  9.810 ms  0/0 (0%)
[  7]  15.00-16.00  sec  0.00 Bytes  0.00 bits/sec  9.810 ms  0/0 (0%)
[  7]  16.00-17.00  sec  0.00 Bytes  0.00 bits/sec  9.810 ms  0/0 (0%)
[  7]  17.00-18.00  sec  0.00 Bytes  0.00 bits/sec  9.810 ms  0/0 (0%)
[  7]  18.00-19.00  sec  0.00 Bytes  0.00 bits/sec  9.810 ms  0/0 (0%)
[  7]  19.00-20.00  sec  0.00 Bytes  0.00 bits/sec  9.810 ms  0/0 (0%)
```

Figura 4.1: Output del server `H3`: ricezione di due flussi simultanei, con il traffico di `H1` limitato/bloccato.

```
"Node: h3"
Server listening on 5201
-----------------------------------------------------------
Accepted connection from 10.0.0.2, port 52634
[  7] local 10.0.0.3 port 5201 connected to 10.0.0.2 port 57096
[ ID] Interval           Transfer     Bitrate         Jitter    Lost/Total Datag
rams
[  7]   0.00-1.00   sec  99.0 KBytes   811 Kbits/sec  0.408 ms  0/70 (0%)
[  7]   1.00-2.00   sec   122 KBytes   996 Kbits/sec  0.384 ms  0/86 (0%)
[  7]   2.00-3.00   sec   123 KBytes  1.01 Mbits/sec  0.341 ms  0/87 (0%)
[  7]   3.00-4.00   sec   122 KBytes   996 Kbits/sec  0.694 ms  0/86 (0%)
[  7]   4.00-5.00   sec   122 KBytes   996 Kbits/sec  0.515 ms  0/86 (0%)
[  7]   5.00-6.00   sec   123 KBytes  1.01 Mbits/sec  0.591 ms  0/87 (0%)
[  7]   6.00-7.00   sec   122 KBytes   997 Kbits/sec  0.837 ms  0/86 (0%)
[  7]   7.00-8.00   sec   122 KBytes   996 Kbits/sec  0.460 ms  0/86 (0%)
[  7]   8.00-9.00   sec   123 KBytes  1.01 Mbits/sec  0.620 ms  0/87 (0%)
[  7]   9.00-10.00  sec   122 KBytes   996 Kbits/sec  0.383 ms  0/86 (0%)
[  7]  10.00-11.00  sec   122 KBytes   996 Kbits/sec  0.522 ms  0/86 (0%)
[  7]  11.00-12.00  sec   123 KBytes  1.01 Mbits/sec  0.625 ms  0/87 (0%)
[  7]  12.00-13.00  sec   122 KBytes   996 Kbits/sec  0.429 ms  0/86 (0%)
[  7]  13.00-14.00  sec   122 KBytes   996 Kbits/sec  0.604 ms  0/86 (0%)
[  7]  14.00-15.00  sec   123 KBytes  1.01 Mbits/sec  0.543 ms  0/87 (0%)
[  7]  15.00-16.00  sec   122 KBytes   996 Kbits/sec  0.236 ms  0/86 (0%)
[  7]  16.00-17.00  sec   122 KBytes   996 Kbits/sec  0.942 ms  0/86 (0%)
[  7]  17.00-18.00  sec   123 KBytes  1.01 Mbits/sec  0.345 ms  0/87 (0%)
[  7]  18.00-19.00  sec   122 KBytes   996 Kbits/sec  0.721 ms  0/86 (0%)
[  7]  19.00-20.00  sec   122 KBytes   996 Kbits/sec  0.366 ms  0/86 (0%)
[  7]  20.00-20.19  sec  24.0 KBytes  1.02 Mbits/sec  0.536 ms  0/17 (0%)
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bitrate         Jitter    Lost/Total Datag
rams
[  7]   0.00-20.19  sec  2.38 MBytes   991 Kbits/sec  0.536 ms  0/1727 (0%)  rec
eiver
```

Figura 4.2: Server `H3`: il flusso legittimo proveniente da `H2` rimane invariato e stabile.

```
"Node: h1"
root@smag01:/home/smag01/vmshare/sdn_defender/topology# iperf3 -c 10.0.0.3 -p 5
201 -u -b 50M -t 20 -i 1
Connecting to host 10.0.0.3, port 5201
[  7] local 10.0.0.1 port 56184 connected to 10.0.0.3 port 5201
[ ID] Interval           Transfer     Bitrate         Total Datagrams
[  7]   0.00-1.00   sec  5.96 MBytes  50.0 Mbits/sec  4314
[  7]   1.00-2.00   sec  5.96 MBytes  50.0 Mbits/sec  4318
[  7]   2.00-3.00   sec  5.96 MBytes  50.0 Mbits/sec  4317
[  7]   3.00-4.00   sec  5.96 MBytes  50.0 Mbits/sec  4313
[  7]   4.00-5.00   sec  5.96 MBytes  50.0 Mbits/sec  4318
[  7]   5.00-6.00   sec  5.96 MBytes  50.0 Mbits/sec  4315
[  7]   6.00-7.00   sec  5.96 MBytes  50.0 Mbits/sec  4316
[  7]   7.00-8.00   sec  5.96 MBytes  50.0 Mbits/sec  4318
[  7]   8.00-9.00   sec  5.96 MBytes  50.0 Mbits/sec  4315
[  7]   9.00-10.00  sec  5.96 MBytes  50.0 Mbits/sec  4319
[  7]  10.00-11.00  sec  5.96 MBytes  50.0 Mbits/sec  4313
[  7]  11.00-12.00  sec  5.96 MBytes  50.0 Mbits/sec  4316
[  7]  12.00-13.00  sec  5.96 MBytes  50.0 Mbits/sec  4317
[  7]  13.00-14.00  sec  5.96 MBytes  50.0 Mbits/sec  4319
[  7]  14.00-15.00  sec  5.96 MBytes  50.0 Mbits/sec  4316
[  7]  15.00-16.00  sec  5.96 MBytes  50.0 Mbits/sec  4315
[  7]  16.00-17.00  sec  5.96 MBytes  50.0 Mbits/sec  4316
[  7]  17.00-18.00  sec  5.96 MBytes  50.0 Mbits/sec  4315
[  7]  18.00-19.00  sec  5.96 MBytes  50.0 Mbits/sec  4318
[  7]  19.00-20.00  sec  5.96 MBytes  50.0 Mbits/sec  4315
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bitrate         Jitter    Lost/Total Datag
rams
[  7]   0.00-20.00  sec   119 MBytes  50.0 Mbits/sec  0.000 ms  0/86323 (0%)  se
nder
[  7]   0.00-20.00  sec  0.00 Bytes  0.00 bits/sec  0.000 ms  0/0 (0%)  receiver
```

Figura 4.3: Terminale di `H1`: generazione del traffico UDP malevolo ad alta intensità.

```
"Node: h2"
root@smag01:/home/smag01/vmshare/sdn_defender/topology# iperf3 -c 10.0.0.3 -p 5
201 -u -b 1M -t 20 -i 1
Connecting to host 10.0.0.3, port 5201
[  7] local 10.0.0.2 port 57096 connected to 10.0.0.3 port 5201
[ ID] Interval           Transfer     Bitrate         Total Datagrams
[  7]   0.00-1.00   sec   123 KBytes  1.01 Mbits/sec  87
[  7]   1.00-2.00   sec   122 KBytes   996 Kbits/sec  86
[  7]   2.00-3.00   sec   122 KBytes   997 Kbits/sec  86
[  7]   3.00-4.00   sec   123 KBytes  1.01 Mbits/sec  87
[  7]   4.00-5.00   sec   122 KBytes   996 Kbits/sec  86
[  7]   5.00-6.00   sec   122 KBytes   996 Kbits/sec  86
[  7]   6.00-7.00   sec   123 KBytes  1.01 Mbits/sec  87
[  7]   7.00-8.00   sec   122 KBytes   996 Kbits/sec  86
[  7]   8.00-9.00   sec   122 KBytes   996 Kbits/sec  86
[  7]   9.00-10.00  sec   123 KBytes  1.01 Mbits/sec  87
[  7]  10.00-11.00  sec   122 KBytes   997 Kbits/sec  86
[  7]  11.00-12.00  sec   122 KBytes   996 Kbits/sec  86
[  7]  12.00-13.00  sec   123 KBytes  1.01 Mbits/sec  87
[  7]  13.00-14.00  sec   122 KBytes   996 Kbits/sec  86
[  7]  14.00-15.00  sec   122 KBytes   996 Kbits/sec  86
[  7]  15.00-16.00  sec   123 KBytes  1.01 Mbits/sec  87
[  7]  16.00-17.00  sec   122 KBytes   996 Kbits/sec  86
[  7]  17.00-18.00  sec   122 KBytes   996 Kbits/sec  86
[  7]  18.00-19.00  sec   123 KBytes  1.01 Mbits/sec  87
[  7]  19.00-20.00  sec   122 KBytes   996 Kbits/sec  86
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bitrate         Jitter    Lost/Total Datag
rams
[  7]   0.00-20.00  sec  2.38 MBytes  1.00 Mbits/sec  0.000 ms  0/1727 (0%)  sen
der
[  7]   0.00-20.19  sec  2.38 MBytes   991 Kbits/sec  0.536 ms  0/1727 (0%)  rec
eiver

iperf Done.
```

Figura 4.4: Terminale di `H2`: generazione del flusso UDP legittimo stabile a 1 Mbit/s.

### 4.2.5 Log del Controller

Il log del controller (Figura 4.5) conferma il corretto comportamento selettivo del sistema. Le righe mostrano come le azioni di `LIMIT` e `BLOCK` vengano applicate esclusivamente al flusso proveniente da `H1` (`eth_src=00:00:00:00:00:01`), senza coinvolgere `H2`.

```
===== STATUS LOG =====
[FLOW] dpid=1, in_port=2, src=00:00:00:00:00:03, dst=00:00:00:00:00:01, rate=0.0
 B/s, dyn_thr=3568.0
[FLOW] dpid=1, in_port=1, src=00:00:00:00:00:01, dst=00:00:00:00:00:03, rate=100
00880.0 B/s, dyn_thr=4533592.0
[FLOW] dpid=3, in_port=3, src=00:00:00:00:00:03, dst=00:00:00:00:00:01, rate=0.0
 B/s, dyn_thr=3568.0
[FLOW] dpid=3, in_port=1, src=00:00:00:00:00:01, dst=00:00:00:00:00:03, rate=601
9600.0 B/s, dyn_thr=2590632.0
[FLOW] dpid=4, in_port=2, src=00:00:00:00:00:03, dst=00:00:00:00:00:01, rate=0.0
 B/s, dyn_thr=3568.0
[FLOW] dpid=4, in_port=1, src=00:00:00:00:00:01, dst=00:00:00:00:00:03, rate=601
9600.0 B/s, dyn_thr=2435672.0
======================

[BURST-STATE] FlowKey(dpid=1, in_port=1, eth_src='00:00:00:00:00:01', eth_dst='0
0:00:00:00:00:03', ip_src=None, ip_dst=None, tp_dst=None) changed to True (mean=
8174477, cv=0.39, max/ewma=1.54)
LIMIT FLOW FlowKey(dpid=1, in_port=1, eth_src='00:00:00:00:00:01', eth_dst='00:0
0:00:00:00:03', ip_src=None, ip_dst=None, tp_dst=None) (RATE) count=0 bps=998896
0 thr=6094954 cv=0.39 peak/ewma=1.54 (ingress=1)
Installing LIMIT 1000 kbps on flow FlowKey(dpid=1, in_port=1, eth_src='00:00:00:
00:00:01', eth_dst='00:00:00:00:00:03', ip_src=None, ip_dst=None, tp_dst=None)
[BURST-STATE] FlowKey(dpid=3, in_port=1, eth_src='00:00:00:00:00:01', eth_dst='0
0:00:00:00:00:03', ip_src=None, ip_dst=None, tp_dst=None) changed to True (mean=
4305116, cv=0.56, max/ewma=1.84)
[BURST-STATE] FlowKey(dpid=4, in_port=1, eth_src='00:00:00:00:00:01', eth_dst='0
0:00:00:00:00:03', ip_src=None, ip_dst=None, tp_dst=None) changed to True (mean=
4227636, cv=0.60, max/ewma=1.91)
[BURST DETECTED] FlowKey(dpid=1, in_port=1, eth_src='00:00:00:00:00:01', eth_dst
='00:00:00:00:00:03', ip_src=None, ip_dst=None, tp_dst=None) mean=1341921 cv=8.5
7 max/ewma=16.76
Installing DROP (flow) FlowKey(dpid=1, in_port=1, eth_src='00:00:00:00:00:01', e
th_dst='00:00:00:00:00:03', ip_src=None, ip_dst=None, tp_dst=None)
[BLOCKED] after 5 consecutive LIMITs FlowKey(dpid=1, in_port=1, eth_src='00:00:0
0:00:00:01', eth_dst='00:00:00:00:00:03', ip_src=None, ip_dst=None, tp_dst=None)
```

Figura 4.5: Log del controller Ryu: azioni di `LIMIT` e `BLOCK` applicate solo al flusso di `H1`.

Le fasi principali osservate sono:

- **Rilevamento:** il controller rileva il flusso di `H1` come anomalo, con un throughput superiore alla soglia di 2 Mbit/s.
- **Mitigazione selettiva:** viene applicata una regola di `LIMIT` pari a 1 Mbit/s sul solo flusso dell'attaccante.
- **Escalation:** dopo 5 tentativi consecutivi di limitazione inefficace, il controller applica il `BLOCK FLOW`.

### 4.2.6 Risultati e Validazione

Il test conferma la risoluzione completa del problema di *overblocking* dovuto a mitigazioni port-level. Il nuovo approccio **flow-level** consente di:

- isolare il flusso malevolo (`H1` $\rightarrow$ `H3:5201`) in modo preciso;
- mantenere intatto il traffico legittimo (`H2` $\rightarrow$ `H3:5201`);
- garantire un comportamento coerente con gli obiettivi di affidabilità e disponibilità della rete.

In conclusione, la mitigazione selettiva introdotta nel modulo `PolicyEngine` dimostra un comportamento granulare ed efficace, evitando l'overblocking e migliorando la qualità complessiva del traffico in presenza di attacchi.

## 4.3 Test: Burst Attack Detection

### Obiettivo

Questo test ha lo scopo di verificare la capacità del controller di rilevare e mitigare attacchi di tipo *burst*, caratterizzati da brevi raffiche di traffico ad alta intensità alternate a periodi di inattività.

### 4.3.1 Scenario di Test

- **Topologia:** la stessa utilizzata nei test precedenti, composta da tre host (`H1`, `H2`, `H3`) e uno switch OpenFlow connesso al controller Ryu.
- **Ruoli:** `H1` agisce da host attaccante, mentre `H3` funge da server vittima in ascolto.
- **Traffico generato:** una sequenza di otto raffiche UDP da 7 Mbit/s, ciascuna della durata di 1 s, alternate a pause di 2 s. Questo comportamento intermittente simula un attacco *burst-like*, in cui il traffico oscilla rapidamente tra picchi e inattività, mascherando temporaneamente il superamento della soglia media.

### 4.3.2 Comandi di esecuzione

Il traffico è stato generato nel terminale Mininet utilizzando i seguenti comandi:

```
mininet> h3 iperf3 -s -p 5201 -D
mininet> h1 bash -c 'for i in $(seq 1 8);  do
    iperf3 -c 10.0.0.3 -u -b 7M -t 1 -p 5201; sleep 2; do
```

In questo modo, `H1` invia pacchetti UDP verso `H3` con bitrate di 7 Mbit/s. Essendo il valore vicino alla soglia di rilevamento (~2 Mbit/s),

le prime raffiche non vengono immediatamente classificate come anomale. Tuttavia, dopo alcuni cicli, l'aumento del coefficiente di variazione (`cv`) e del rapporto `peak/ewma` porta il controller a riconoscere il comportamento burst e ad applicare le azioni di mitigazione.

### 4.3.3 Evidenze sperimentali

Durante l'esecuzione, il controller ha monitorato il traffico calcolando continuamente i valori di:

- `cv` (*Coefficient of Variation*) — misura la variabilità del throughput;
- `peak/ewma` — rapporto tra il picco istantaneo e la media esponenziale mobile.

Il log riportato in Figura 4.6 mostra la sequenza di eventi: inizialmente i valori di `cv` e `ewma` sono bassi, poi aumentano progressivamente con il susseguirsi delle raffiche. Ciò causa il passaggio degli stati di rilevamento (`BURST-STATE`) e infine l'attivazione della mitigazione con messaggi di `BURST DETECTED`.

```
smag01@smag01: ~/vmshare

===== STATUS LOG =====
======================


===== STATUS LOG =====
[FLOW] dpid=4, in_port=2, src=00:00:00:00:00:03, dst=00:00:00:00:00:01, rate=0.0 B/s, dyn_thr=0.0
[FLOW] dpid=4, in_port=1, src=00:00:00:00:00:01, dst=00:00:00:00:00:03, rate=5161888.0 B/s, dyn_thr
=0.0
[FLOW] dpid=1, in_port=2, src=00:00:00:00:00:03, dst=00:00:00:00:00:01, rate=912.0 B/s, dyn_thr=0.0
[FLOW] dpid=1, in_port=1, src=00:00:00:00:00:01, dst=00:00:00:00:00:03, rate=6914128.0 B/s, dyn_thr
=0.0
[FLOW] dpid=3, in_port=3, src=00:00:00:00:00:03, dst=00:00:00:00:00:01, rate=544.0 B/s, dyn_thr=0.0
[FLOW] dpid=3, in_port=1, src=00:00:00:00:00:01, dst=00:00:00:00:00:03, rate=5316848.0 B/s, dyn_thr
=0.0
======================

[BURST-STATE] FlowKey(dpid=4, in_port=2, eth_src='00:00:00:00:00:03', eth_dst='00:00:00:00:00:01',
ip_src=None, ip_dst=None, tp_dst=None) changed to False (mean=0, cv=0.00, max/ewma=0.00)
[BURST-STATE] FlowKey(dpid=4, in_port=1, eth_src='00:00:00:00:00:01', eth_dst='00:00:00:00:00:03',
ip_src=None, ip_dst=None, tp_dst=None) changed to False (mean=5161888, cv=0.00, max/ewma=1.00)
[BURST-STATE] FlowKey(dpid=1, in_port=2, eth_src='00:00:00:00:00:03', eth_dst='00:00:00:00:00:01',
ip_src=None, ip_dst=None, tp_dst=None) changed to False (mean=912, cv=0.00, max/ewma=1.00)
[BURST-STATE] FlowKey(dpid=1, in_port=1, eth_src='00:00:00:00:00:01', eth_dst='00:00:00:00:00:03',
ip_src=None, ip_dst=None, tp_dst=None) changed to False (mean=6914128, cv=0.00, max/ewma=1.00)
[BURST-STATE] FlowKey(dpid=3, in_port=3, eth_src='00:00:00:00:00:03', eth_dst='00:00:00:00:00:01',
ip_src=None, ip_dst=None, tp_dst=None) changed to False (mean=544, cv=0.00, max/ewma=1.00)
[BURST-STATE] FlowKey(dpid=3, in_port=1, eth_src='00:00:00:00:00:01', eth_dst='00:00:00:00:00:03',
ip_src=None, ip_dst=None, tp_dst=None) changed to False (mean=5316848, cv=0.00, max/ewma=1.00)
[BURST-STATE] FlowKey(dpid=1, in_port=1, eth_src='00:00:00:00:00:01', eth_dst='00:00:00:00:00:03',
ip_src=None, ip_dst=None, tp_dst=None) changed to True (mean=2402362, cv=1.63, max/ewma=1.55)
[BURST DETECTED] FlowKey(dpid=1, in_port=1, eth_src='00:00:00:00:00:01', eth_dst='00:00:00:00:00:03
', ip_src=None, ip_dst=None, tp_dst=None) mean=2402362 cv=1.63 max/ewma=1.55
[BURST DETECTED] FlowKey(dpid=1, in_port=1, eth_src='00:00:00:00:00:01', eth_dst='00:00:00:00:00:03
', ip_src=None, ip_dst=None, tp_dst=None) mean=1802530 cv=1.89 max/ewma=1.93
[BURST-STATE] FlowKey(dpid=4, in_port=1, eth_src='00:00:00:00:00:01', eth_dst='00:00:00:00:00:03',
ip_src=None, ip_dst=None, tp_dst=None) changed to True (mean=2126809, cv=1.05, max/ewma=1.72)
[BURST DETECTED] FlowKey(dpid=4, in_port=1, eth_src='00:00:00:00:00:01', eth_dst='00:00:00:00:00:03
', ip_src=None, ip_dst=None, tp_dst=None) mean=2126809 cv=1.05 max/ewma=1.72
[BURST DETECTED] FlowKey(dpid=1, in_port=1, eth_src='00:00:00:00:00:01', eth_dst='00:00:00:00:00:03
', ip_src=None, ip_dst=None, tp_dst=None) mean=2336601 cv=1.36 max/ewma=1.84
[BURST-STATE] FlowKey(dpid=3, in_port=1, eth_src='00:00:00:00:00:01', eth_dst='00:00:00:00:00:03',
ip_src=None, ip_dst=None, tp_dst=None) changed to True (mean=1802258, cv=1.39, max/ewma=1.79)
[BURST DETECTED] FlowKey(dpid=3, in_port=1, eth_src='00:00:00:00:00:01', eth_dst='00:00:00:00:00:03
', ip_src=None, ip_dst=None, tp_dst=None) mean=1802258 cv=1.39 max/ewma=1.79
[BURST DETECTED] FlowKey(dpid=4, in_port=1, eth_src='00:00:00:00:00:01', eth_dst='00:00:00:00:00:03
', ip_src=None, ip_dst=None, tp_dst=None) mean=2402818 cv=0.88 max/ewma=1.63
[BURST DETECTED] FlowKey(dpid=1, in_port=1, eth_src='00:00:00:00:00:01', eth_dst='00:00:00:00:00:03
', ip_src=None, ip_dst=None, tp_dst=None) mean=2336601 cv=1.36 max/ewma=1.84
```

Figura 4.6: Log del controller Ryu durante il test *burst*: progressione degli stati di monitoraggio e rilevamento (`BURST-STATE` e `BURST DETECTED`) in seguito a variazioni del throughput.

Dall'analisi del log emergono le seguenti fasi:

- **Fase iniziale:** le prime raffiche non generano allerta poiché la media mobile non supera la soglia.
- **Aumento della variabilità:** con l'accumulo delle raffiche, i valori di `cv` superano 1.5 e `peak/ewma` cresce oltre 5.0, portando il controller a riconoscere un pattern anomalo.

- **Rilevamento burst:** viene emesso lo stato `BURST DETECTED` sul flusso di `H1`, evidenziando un comportamento tipico di attacco intermittente.

- **Mitigazione selettiva:** in risposta, il modulo `PolicyEngine` applica la regola di `LIMIT` a 1 Mbit/s e, in caso di raffiche ripetute, un `BLOCK FLOW` temporaneo seguito dal successivo `UNBLOCK` in fase di cooldown.

### 4.3.4 Risultati e validazione

Il test dimostra la piena efficacia del meccanismo di rilevamento dinamico implementato nel modulo `PolicyEngine`. Il controller non si basa esclusivamente sul valore assoluto del throughput, ma adotta un'analisi statistica temporale in grado di identificare pattern *burst-like* anche a intensità moderata. L'approccio consente di:

- rilevare traffico anomalo con oscillazioni irregolari di breve durata;

- attivare mitigazioni proporzionate e temporanee;

- preservare il traffico legittimo e ridurre i falsi positivi.

In sintesi, il sistema SDN Defender reagisce in modo adattivo e progressivo, confermando la robustezza dell'algoritmo di **Burst Detection** nella protezione della rete da attacchi intermittenti.

## 4.4 Test: Funzionalità di Blackhole

### 4.4.1 Obiettivo

Questo test ha lo scopo di validare il corretto funzionamento del meccanismo di **Blackhole** introdotto nel controller, progettato per bloccare temporaneamente tutto il traffico destinato a un host vittima in presenza di un attacco *distributed flood*. L'obiettivo è dimostrare che il controller:

1. rileva un attacco distribuito proveniente da più sorgenti (multi-switch);
2. installa regole di `DROP` temporanee su tutti gli switch coinvolti;
3. rimuove automaticamente tali regole dopo la finestra temporale configurata, ristabilendo la connettività.

### 4.4.2 Scenario di Test

- **Topologia:** rete composta da quattro host (`H1-H4`) e quattro switch OpenFlow collegati al controller Ryu. L'host `H3` funge da vittima, mentre `H1`, `H2` e `H4` agiscono come sorgenti di traffico.
- **Traffico generato:**
  - `H1` → `H3`: flusso UDP malevolo ad alta intensità (50 Mbit/s);

  - `H2` → `H3` e `H4` → `H3`: flussi UDP legittimi a bassa intensità (1 Mbit/s ciascuno).

- **Parametri di mitigazione:** finestra di blocco (`BLACKHOLE_TIME`) pari a 20 s, con soglia di attivazione basata sul throughput aggregato e sul numero minimo di switch sorgenti coinvolti.

### 4.4.3 Comandi di esecuzione

Il traffico è stato generato nei terminali associati ai singoli host con i seguenti comandi:

```
# Su H3 (server)
iperf3 -s -p 5201 -D
iperf3 -s -p 5202 -D
iperf3 -s -p 5203 -D

# Su H1 (attacco principale)
iperf3 -c 10.0.0.3 -p 5201 -u -b 50M -t 30 -i 1

# Su H2 (flusso legittimo)
iperf3 -c 10.0.0.3 -p 5202 -u -b 1M -t 30 -i 1

# Su H4 (flusso legittimo)
iperf3 -c 10.0.0.3 -p 5203 -u -b 1M -t 30 -i 1
```

Durante il test, `H3` riceve simultaneamente tre flussi UDP con differente intensità di banda. Il controller rileva l'anomalia aggregata e attiva il meccanismo di `BLACKHOLE` su tutti gli switch che inoltrano pacchetti verso la vittima.

### 4.4.4 Evidenze sperimentali

Le Figure 4.7–4.9 mostrano le evidenze raccolte durante l'esperimento.

```
"Node: h3"
Accepted connection from 10.0.0.1, port 49452
[  7] local 10.0.0.3 port 5201 connected to 10.0.0.1 port 40304
[ ID] Interval           Transfer     Bitrate         Jitter    Lost/Total Datag
rams
[  7]   0.00-1.00   sec   612 KBytes  5.02 Mbits/sec  1.776 ms  0/433 (0%)
[  7]   1.00-2.00   sec   711 KBytes  5.82 Mbits/sec  1.790 ms  0/503 (0%)
[  7]   2.00-3.00   sec   711 KBytes  5.83 Mbits/sec  1.039 ms  809/1312 (62%)
Accepted connection from 10.0.0.2, port 39310
[  7] local 10.0.0.3 port 5203 connected to 10.0.0.2 port 45451
Accepted connection from 10.0.0.4, port 37054
[  7] local 10.0.0.3 port 5202 connected to 10.0.0.4 port 41910
[  7]   3.00-4.00   sec   503 KBytes  4.12 Mbits/sec  2.962 ms  8850/9206 (96%)

[ ID] Interval           Transfer     Bitrate         Jitter    Lost/Total Datag
rams
[  7]   0.00-1.00   sec  99.0 KBytes   811 Kbits/sec  0.933 ms  0/70 (0%)
[ ID] Interval           Transfer     Bitrate         Jitter    Lost/Total Datag
rams
[  7]   0.00-1.00   sec   106 KBytes   869 Kbits/sec  0.557 ms  0/75 (0%)
[  7]   4.00-5.00   sec   119 KBytes   973 Kbits/sec  0.879 ms  4240/4324 (98%)

[  7]   1.00-2.00   sec   123 KBytes  1.01 Mbits/sec  0.684 ms  0/87 (0%)
[  7]   1.00-2.00   sec   122 KBytes   996 Kbits/sec  0.491 ms  0/86 (0%)
[  7]   5.00-6.00   sec   119 KBytes   973 Kbits/sec  0.765 ms  4230/4314 (98%)

[  7]   2.00-3.00   sec   122 KBytes   996 Kbits/sec  0.698 ms  0/86 (0%)
[  7]   2.00-3.00   sec   122 KBytes   997 Kbits/sec  0.560 ms  0/86 (0%)
[  7]   6.00-7.00   sec   113 KBytes   927 Kbits/sec  0.675 ms  4042/4122 (98%)

[  7]   3.00-4.00   sec  97.6 KBytes   799 Kbits/sec  0.637 ms  0/69 (0%)
[  7]   3.00-4.00   sec  17.0 KBytes   139 Kbits/sec  0.554 ms  0/12 (0%)
[  7]   7.00-8.00   sec  0.00 Bytes  0.00 bits/sec  0.675 ms  0/0 (0%)
[  7]   4.00-5.00   sec  0.00 Bytes  0.00 bits/sec  0.637 ms  0/0 (0%)
[  7]   4.00-5.00   sec  0.00 Bytes  0.00 bits/sec  0.554 ms  0/0 (0%)
[  7]   8.00-9.00   sec  0.00 Bytes  0.00 bits/sec  0.675 ms  0/0 (0%)
[  7]   5.00-6.00   sec  0.00 Bytes  0.00 bits/sec  0.637 ms  0/0 (0%)
[  7]   5.00-6.00   sec  0.00 Bytes  0.00 bits/sec  0.554 ms  0/0 (0%)
[  7]   9.00-10.00  sec  0.00 Bytes  0.00 bits/sec  0.675 ms  0/0 (0%)
[  7]   6.00-7.00   sec  0.00 Bytes  0.00 bits/sec  0.637 ms  0/0 (0%)
[  7]   6.00-7.00   sec  0.00 Bytes  0.00 bits/sec  0.554 ms  0/0 (0%)
[  7]  10.00-11.00  sec  0.00 Bytes  0.00 bits/sec  0.675 ms  0/0 (0%)
[  7]   7.00-8.00   sec  0.00 Bytes  0.00 bits/sec  0.637 ms  0/0 (0%)
[  7]   7.00-8.00   sec  0.00 Bytes  0.00 bits/sec  0.554 ms  0/0 (0%)
[  7]  11.00-12.00  sec  0.00 Bytes  0.00 bits/sec  0.675 ms  0/0 (0%)
[  7]   8.00-9.00   sec  0.00 Bytes  0.00 bits/sec  0.637 ms  0/0 (0%)
[  7]   8.00-9.00   sec  0.00 Bytes  0.00 bits/sec  0.554 ms  0/0 (0%)
[  7]  12.00-13.00  sec  0.00 Bytes  0.00 bits/sec  0.675 ms  0/0 (0%)
[  7]   9.00-10.00  sec  0.00 Bytes  0.00 bits/sec  0.637 ms  0/0 (0%)
[  7]   9.00-10.00  sec  0.00 Bytes  0.00 bits/sec  0.554 ms  0/0 (0%)
[  7]  13.00-14.00  sec  0.00 Bytes  0.00 bits/sec  0.675 ms  0/0 (0%)
```

Figura 4.7: Output del server `H3`: ricezione dei flussi concorrenti. Si osserva il traffico proveniente da `H1` (50 Mbit/s) insieme ai flussi di `H2` e `H4` (1 Mbit/s ciascuno). Dopo l'attivazione del `BLACKHOLE`, la ricezione si interrompe temporaneamente.

```
[DDoS] BLACKHOLE dst=00:00:00:00:00:03 total_bps=36660576 sources=3 dpids=2 for 20s
Installing BLACKHOLE on dpid=1 dst=00:00:00:00:00:03 for 20s
Installing BLACKHOLE on dpid=2 dst=00:00:00:00:00:03 for 20s
Installing BLACKHOLE on dpid=3 dst=00:00:00:00:00:03 for 20s
Installing BLACKHOLE on dpid=4 dst=00:00:00:00:00:03 for 20s
```

Figura 4.8: Log del controller Ryu: rilevamento dell'attacco distribuito e installazione delle regole di `BLACKHOLE` su tutti gli switch coinvolti (`dpid=1-4`) per 20 s.

```
[UNBLACKHOLE] dst=00:00:00:00:00:03 expired after cooldown
[DDoS] BLACKHOLE REMOVED  on dpid=4, dst=00:00:00:00:00:03, dur=20.1s, pkts=122, bytes=181780
[DDoS] BLACKHOLE REMOVED  on dpid=2, dst=00:00:00:00:00:03, dur=20.1s, pkts=1454, bytes=2160768
[DDoS] BLACKHOLE REMOVED  on dpid=1, dst=00:00:00:00:00:03, dur=20.1s, pkts=13854, bytes=20635345
[DDoS] BLACKHOLE REMOVED  on dpid=3, dst=00:00:00:00:00:03, dur=20.1s, pkts=122, bytes=181780
```

Figura 4.9: Log del controller Ryu: rimozione automatica delle regole di `BLACKHOLE` al termine del periodo di blocco (`BLACKHOLE REMOVED`) e ripristino della connettività verso la vittima.

Dall'analisi dei log e degli output si osserva che:

- il controller aggrega correttamente il traffico proveniente da più sorgenti e riconosce un pattern di attacco distribuito;
- vengono installate regole di `DROP` (blackhole) su tutti gli switch coinvolti nella catena di forwarding;
- la durata effettiva del blocco è di circa 20 s, come configurato;
- al termine, le regole vengono automaticamente rimosse e la normale comunicazione verso `H3` viene ripristinata;

### 4.4.5 Risultati e validazione

Il test conferma il corretto funzionamento del meccanismo di `BLACKHOLE` nel controller SDN Defender. Il sistema è in grado di:

- rilevare in modo cooperativo un attacco *distributed flood* su più switch;
- applicare la mitigazione in modo coordinato e temporaneo;
- ristabilire automaticamente la connettività al termine del periodo di blocco.

Il comportamento osservato è coerente con le specifiche progettuali: il blocco è **temporaneo, selettivo e multi-switch**, garantendo la continuità del servizio per i flussi legittimi e la rapida rimozione della minaccia dopo il cooldown.

### 4.4.6 Criteri di superamento

Il test è considerato superato se:

- il meccanismo di `BLACKHOLE` si attiva solo in presenza di un attacco distribuito e di soglia aggregata superata;
- le regole di drop vengono propagate su tutti gli switch coinvolti e rimosse automaticamente dopo 20 s;

## 4.5 Test: Flexible Blocking / Unblocking Policy

### 4.5.1 Obiettivo

Questa sezione raccoglie i test eseguiti per validare la nuova **politica di blocco e sblocco flessibile** introdotta nel controller SDN Defender, progettata per risolvere tre criticità del sistema precedente:

1. blocchi permanenti anche dopo la cessazione del traffico anomalo (assenza di cooldown automatico);
2. impossibilità di escludere host affidabili dalla mitigazione (assenza di whitelist MAC);
3. assenza di controllo manuale delle regole di blocco/sblocco tramite API REST (blocklist dinamica).

Gli esperimenti mostrano che il nuovo sistema gestisce dinamicamente le regole di enforcement, adattandosi in tempo reale al comportamento del traffico e ripristinando automaticamente la connettività una volta che la minaccia si riduce.

### 4.5.2 Test 1 – Automatic Unblock e Cooldown dinamico

**Scenario di Test.** Questo test verifica il corretto funzionamento del meccanismo di **cooldown**, che consente di rimuovere automaticamente le regole di blocco quando il traffico anomalo ritorna sotto la soglia di allarme. Il controller applica inizialmente un blocco temporaneo al flusso anomalo e, una volta rilevata la stabilizzazione del throughput, esegue lo sblocco automatico (`UNBLOCK FLOW`). Se il flusso rimane attivo e successivamente supera di nuovo la soglia, la regola di blocco viene reinstallata.

**Comandi di esecuzione:**

```
mininet> h3 iperf3 -s -p 5201 -D
mininet> h1 iperf3 -c 10.0.0.3 -p 5201 -u -b 50M -t 150 -
```

In questo scenario:

- `H3` funge da server UDP sulla porta 5201;
- `H1` genera un flusso costante di 50 Mbit/s per 150 s, sufficiente per innescare più cicli di blocco/sblocco;
- il controller monitora il traffico e applica automaticamente `LIMIT` e `BLOCK` al superamento della soglia configurata (≈2 Mbit/s).

### 4.5.3 Evidenze sperimentali

Le Figure 4.10–4.13 mostrano i risultati ottenuti durante l'esperimento.

```
"Node: h3"
root@smag01:/home/smag01/vmshare/sdn_defender/topology# iperf3 -s -p 5201 &
[1] 2590
root@smag01:/home/smag01/vmshare/sdn_defender/topology# ------------------------
----------------------------------
Server listening on 5201
-----------------------------------------------------------
Accepted connection from 10.0.0.1, port 56842
  [  7] local 10.0.0.3 port 5201 connected to 10.0.0.1 port 52133
  [ ID] Interval           Transfer     Bitrate         Jitter    Lost/Total Datag
rams
  [  7]   0.00-1.00   sec   611 KBytes  5.00 Mbits/sec  1.751 ms  0/432 (0%)
  [  7]   1.00-2.00   sec   713 KBytes  5.84 Mbits/sec  1.788 ms  0/504 (0%)
  [  7]   2.00-3.00   sec   711 KBytes  5.82 Mbits/sec  0.869 ms  852/1355 (63%)
  [  7]   3.00-4.00   sec   711 KBytes  5.83 Mbits/sec  0.885 ms  2102/2605 (81%)

  [  7]   4.00-5.00   sec   711 KBytes  5.83 Mbits/sec  8.694 ms  3501/4004 (87%)

  [  7]   5.00-6.00   sec  62.2 KBytes   509 Kbits/sec  9.884 ms  2217/2261 (98%)

  [  7]   6.00-7.00   sec  0.00 Bytes  0.00 bits/sec  9.884 ms  0/0 (0%)
  [  7]   7.00-8.00   sec  0.00 Bytes  0.00 bits/sec  9.884 ms  0/0 (0%)
  [  7]   8.00-9.00   sec  0.00 Bytes  0.00 bits/sec  9.884 ms  0/0 (0%)
  [  7]   9.00-10.00  sec  0.00 Bytes  0.00 bits/sec  9.884 ms  0/0 (0%)
  [  7]  10.00-11.00  sec  0.00 Bytes  0.00 bits/sec  9.884 ms  0/0 (0%)
  [  7]  11.00-12.00  sec  0.00 Bytes  0.00 bits/sec  9.884 ms  0/0 (0%)
  [  7]  12.00-13.00  sec  0.00 Bytes  0.00 bits/sec  9.884 ms  0/0 (0%)
  [  7]  13.00-14.00  sec  0.00 Bytes  0.00 bits/sec  9.884 ms  0/0 (0%)
  [  7]  14.00-15.00  sec  0.00 Bytes  0.00 bits/sec  9.884 ms  0/0 (0%)
  [  7]  15.00-16.00  sec  0.00 Bytes  0.00 bits/sec  9.884 ms  0/0 (0%)
  [  7]  16.00-17.00  sec  0.00 Bytes  0.00 bits/sec  9.884 ms  0/0 (0%)
  [  7]  17.00-18.00  sec  0.00 Bytes  0.00 bits/sec  9.884 ms  0/0 (0%)
  [  7]  18.00-19.00  sec  0.00 Bytes  0.00 bits/sec  9.884 ms  0/0 (0%)
  [  7]  19.00-20.00  sec  0.00 Bytes  0.00 bits/sec  9.884 ms  0/0 (0%)
```

Figura 4.10: Output del server `H3`: il traffico proveniente da `H1` (50 Mbit/s) viene inizialmente ricevuto regolarmente, per poi essere interrotto dopo l'attivazione del blocco.

```
"Node: h3"
[  7] 102.00-103.00 sec  0.00 Bytes  0.00 bits/sec  9.884 ms  0/0 (0%)
[  7] 103.00-104.00 sec  0.00 Bytes  0.00 bits/sec  9.884 ms  0/0 (0%)
[  7] 104.00-105.00 sec  0.00 Bytes  0.00 bits/sec  9.884 ms  0/0 (0%)
[  7] 105.00-106.00 sec  0.00 Bytes  0.00 bits/sec  9.884 ms  0/0 (0%)
[  7] 106.00-107.00 sec  0.00 Bytes  0.00 bits/sec  9.884 ms  0/0 (0%)
[  7] 107.00-108.00 sec  0.00 Bytes  0.00 bits/sec  9.884 ms  0/0 (0%)
[  7] 108.00-109.00 sec  0.00 Bytes  0.00 bits/sec  9.884 ms  0/0 (0%)
[  7] 109.00-110.00 sec  0.00 Bytes  0.00 bits/sec  9.884 ms  0/0 (0%)
[  7] 110.00-111.00 sec  0.00 Bytes  0.00 bits/sec  9.884 ms  0/0 (0%)
[  7] 111.00-112.00 sec  0.00 Bytes  0.00 bits/sec  9.884 ms  0/0 (0%)
[  7] 112.00-113.00 sec  0.00 Bytes  0.00 bits/sec  9.884 ms  0/0 (0%)
[  7] 113.00-114.00 sec  0.00 Bytes  0.00 bits/sec  9.884 ms  0/0 (0%)
[  7] 114.00-115.00 sec  0.00 Bytes  0.00 bits/sec  9.884 ms  0/0 (0%)
[  7] 115.00-116.00 sec  0.00 Bytes  0.00 bits/sec  9.884 ms  0/0 (0%)
[  7] 116.00-117.00 sec  0.00 Bytes  0.00 bits/sec  9.884 ms  0/0 (0%)
[  7] 117.00-118.00 sec  0.00 Bytes  0.00 bits/sec  9.884 ms  0/0 (0%)
[  7] 118.00-119.00 sec  0.00 Bytes  0.00 bits/sec  9.884 ms  0/0 (0%)
[  7] 119.00-120.00 sec  0.00 Bytes  0.00 bits/sec  9.884 ms  0/0 (0%)
[  7] 120.00-121.00 sec  0.00 Bytes  0.00 bits/sec  9.884 ms  0/0 (0%)
[  7] 121.00-122.00 sec  0.00 Bytes  0.00 bits/sec  9.884 ms  0/0 (0%)
[  7] 122.00-123.00 sec  0.00 Bytes  0.00 bits/sec  9.884 ms  0/0 (0%)
[  7] 123.00-124.00 sec   604 KBytes  4.95 Mbits/sec  0.863 ms  516387/516814 (1
e+02%)
[  7] 124.00-125.00 sec   592 KBytes  4.85 Mbits/sec  0.798 ms  1736/2155 (81%)

[  7] 125.00-126.00 sec  0.00 Bytes  0.00 bits/sec  0.798 ms  0/0 (0%)
[  7] 126.00-127.00 sec  0.00 Bytes  0.00 bits/sec  0.798 ms  0/0 (0%)
[  7] 127.00-128.00 sec  0.00 Bytes  0.00 bits/sec  0.798 ms  0/0 (0%)
[  7] 128.00-129.00 sec  0.00 Bytes  0.00 bits/sec  0.798 ms  0/0 (0%)
[  7] 129.00-130.00 sec  0.00 Bytes  0.00 bits/sec  0.798 ms  0/0 (0%)
[  7] 130.00-131.00 sec  0.00 Bytes  0.00 bits/sec  0.798 ms  0/0 (0%)
[  7] 131.00-132.00 sec  0.00 Bytes  0.00 bits/sec  0.798 ms  0/0 (0%)
[  7] 132.00-133.00 sec  0.00 Bytes  0.00 bits/sec  0.798 ms  0/0 (0%)
[  7] 133.00-134.00 sec  0.00 Bytes  0.00 bits/sec  0.798 ms  0/0 (0%)
[  7] 134.00-135.00 sec  0.00 Bytes  0.00 bits/sec  0.798 ms  0/0 (0%)
[  7] 135.00-136.00 sec  0.00 Bytes  0.00 bits/sec  0.798 ms  0/0 (0%)
[  7] 136.00-137.00 sec  0.00 Bytes  0.00 bits/sec  0.798 ms  0/0 (0%)
```

Figura 4.11: Server `H3`: dopo la scadenza del timer di blocco (`cooldown`), la ricezione riprende automaticamente, ma il traffico rimane anomalo e il controller riapplica una nuova regola di blocco.

```
===== STATUS LOG =====
[FLOW] dpid=1, in_port=2, src=00:00:00:00:00:03, dst=00:00:00:00:00:01, rate=0.0 B/s, d
yn_thr=3568.0
[FLOW] dpid=1, in_port=1, src=00:00:00:00:00:01, dst=00:00:00:00:00:03, rate=9941280.0
B/s, dyn_thr=4486744.0
[FLOW] dpid=3, in_port=3, src=00:00:00:00:00:03, dst=00:00:00:00:00:01, rate=0.0 B/s, d
yn_thr=3568.0
[FLOW] dpid=3, in_port=1, src=00:00:00:00:00:01, dst=00:00:00:00:00:03, rate=6019600.0
B/s, dyn_thr=2590640.0
[FLOW] dpid=4, in_port=2, src=00:00:00:00:00:03, dst=00:00:00:00:00:01, rate=0.0 B/s, d
yn_thr=3568.0
[FLOW] dpid=4, in_port=1, src=00:00:00:00:00:01, dst=00:00:00:00:00:03, rate=6019600.0
B/s, dyn_thr=2435680.0
======================

[BURST-STATE] FlowKey(dpid=1, in_port=1, eth_src='00:00:00:00:00:01', eth_dst='00:00:00
:00:00:03', ip_src=None, ip_dst=None, tp_dst=None) changed to True (mean=7214012, cv=0.
53, max/ewma=1.78)
LIMIT FLOW FlowKey(dpid=1, in_port=1, eth_src='00:00:00:00:00:01', eth_dst='00:00:00:00
:00:03', ip_src=None, ip_dst=None, tp_dst=None) (RATE) count=0 bps=9941280 thr=6038637
cv=0.53 peak/ewma=1.78 (ingress=1)
Installing LIMIT 1000 kbps on flow FlowKey(dpid=1, in_port=1, eth_src='00:00:00:00:00:0
1', eth_dst='00:00:00:00:00:03', ip_src=None, ip_dst=None, tp_dst=None)
[BURST-STATE] FlowKey(dpid=3, in_port=1, eth_src='00:00:00:00:00:01', eth_dst='00:00:00
:00:00:03', ip_src=None, ip_dst=None, tp_dst=None) changed to True (mean=4305120, cv=0.
56, max/ewma=1.84)
[BURST-STATE] FlowKey(dpid=4, in_port=1, eth_src='00:00:00:00:00:01', eth_dst='00:00:00
:00:00:03', ip_src=None, ip_dst=None, tp_dst=None) changed to True (mean=4227640, cv=0.
60, max/ewma=1.91)
[BURST DETECTED] FlowKey(dpid=1, in_port=1, eth_src='00:00:00:00:00:01', eth_dst='00:00
:00:00:00:03', ip_src=None, ip_dst=None, tp_dst=None) mean=1663926 cv=6.47 max/ewma=10.
90
[BURST-STATE] FlowKey(dpid=3, in_port=1, eth_src='00:00:00:00:00:01', eth_dst='00:00:00
:00:00:03', ip_src=None, ip_dst=None, tp_dst=None) changed to False (mean=4912040, cv=0
.33, max/ewma=1.48)
Installing DROP (flow) FlowKey(dpid=1, in_port=1, eth_src='00:00:00:00:00:01', eth_dst=
'00:00:00:00:00:03', ip_src=None, ip_dst=None, tp_dst=None)
[BLOCKED] after 5 consecutive LIMITs FlowKey(dpid=1, in_port=1, eth_src='00:00:00:00:00
:01', eth_dst='00:00:00:00:00:03', ip_src=None, ip_dst=None, tp_dst=None)
[BURST-STATE] FlowKey(dpid=4, in_port=1, eth_src='00:00:00:00:00:01', eth_dst='00:00:00
:00:00:03', ip_src=None, ip_dst=None, tp_dst=None) changed to False (mean=5097792, cv=0
.30, max/ewma=1.37)
```

Figura 4.12: Log del controller Ryu: rilevamento del flusso anomalo, superamento soglia e applicazione della regola di `BLOCK FLOW`.

```
===== STATUS LOG =====
[FLOW] dpid=1, in_port=2, src=00:00:00:00:00:03, dst=00:00:00:00:00:01, rate=0.0 B/s, d
yn_thr=15.4
[FLOW] dpid=1, in_port=1, src=00:00:00:00:00:01, dst=00:00:00:00:00:03, rate=0.0 B/s, d
yn_thr=25.2
[FLOW] dpid=3, in_port=3, src=00:00:00:00:00:03, dst=00:00:00:00:00:01, rate=0.0 B/s, d
yn_thr=15.4
[FLOW] dpid=3, in_port=1, src=00:00:00:00:00:01, dst=00:00:00:00:00:03, rate=0.0 B/s, d
yn_thr=17.2
[FLOW] dpid=4, in_port=2, src=00:00:00:00:00:03, dst=00:00:00:00:00:01, rate=0.0 B/s, d
yn_thr=15.4
[FLOW] dpid=4, in_port=1, src=00:00:00:00:00:01, dst=00:00:00:00:00:03, rate=0.0 B/s, d
yn_thr=16.7
======================

UNBLOCK FLOW (cooldown) FlowKey(dpid=1, in_port=1, eth_src='00:00:00:00:00:01', eth_dst
='00:00:00:00:00:03', ip_src=None, ip_dst=None, tp_dst=None)
Removing DROP (flow) FlowKey(dpid=1, in_port=1, eth_src='00:00:00:00:00:01', eth_dst='0
0:00:00:00:00:03', ip_src=None, ip_dst=None, tp_dst=None)
LIMIT FLOW FlowKey(dpid=1, in_port=1, eth_src='00:00:00:00:00:01', eth_dst='00:00:00:00
:00:03', ip_src=None, ip_dst=None, tp_dst=None) (RATE) count=3 bps=2908480 thr=2000000
cv=3.16 peak/ewma=5.00 (ingress=1)
Installing LIMIT 1000 kbps on flow FlowKey(dpid=1, in_port=1, eth_src='00:00:00:00:00:0
1', eth_dst='00:00:00:00:00:03', ip_src=None, ip_dst=None, tp_dst=None)
Installing DROP (flow) FlowKey(dpid=1, in_port=1, eth_src='00:00:00:00:00:01', eth_dst=
'00:00:00:00:00:03', ip_src=None, ip_dst=None, tp_dst=None)
[BLOCKED] after 5 consecutive LIMITs FlowKey(dpid=1, in_port=1, eth_src='00:00:00:00:00
:01', eth_dst='00:00:00:00:00:03', ip_src=None, ip_dst=None, tp_dst=None)
```

Figura 4.13: Log del controller Ryu: rimozione automatica del blocco (`UNBLOCK FLOW (cooldown)`) seguita da una nuova azione di `BLOCK` sullo stesso flusso, poiché l'attacco persiste.

Dall'analisi dei log e dei dati si osserva che:

- il controller rileva correttamente il traffico anomalo di `H1` e applica la mitigazione dopo alcuni secondi;
- una volta trascorso il periodo di blocco, il modulo di `cooldown` rimuove automaticamente la regola di `DROP`;
- poiché il flusso malevolo resta attivo, il controller riapplica il blocco in modo coerente;
- il ciclo di blocco e sblocco si ripete dinamicamente fino alla cessazione del traffico anomalo.

### 4.5.4 Risultati e validazione

Il test conferma il corretto funzionamento del meccanismo di `cooldown` e della logica di sblocco automatico. Il controller SDN Defender dimostra di poter:

- applicare blocchi temporanei ai flussi anomali e rimuoverli in modo autonomo;
- reagire dinamicamente a traffico persistente reinstallando le regole di `BLOCK`;
- mantenere la continuità dei flussi legittimi, senza generare overblocking.

Il comportamento osservato dimostra una gestione **intelligente e adattiva** del ciclo di enforcement, fondamentale per garantire la stabilità e la resilienza della rete in scenari di traffico variabile.

### 4.5.5 Test 2 – Whitelist MAC (Host affidabili)

**Scenario di Test.** Questo esperimento verifica il corretto funzionamento della **whitelist MAC**, che consente di escludere host affidabili dalle politiche di mitigazione automatica. In particolare, viene mostrato che un host precedentemente soggetto a blocco può essere successivamente "autorizzato" tramite whitelist, continuando a trasmettere senza subire limitazioni.

La funzionalità è stata implementata attraverso un insieme di **alias Bash** definiti nel file `.bashrc`, che permettono di interagire direttamente con le API REST del controller. Digitando il comando:

```
sdnhelp
```

viene mostrata la lista completa delle operazioni disponibili, come riportato in Figura 4.14.

```
smag01@smag01:~$ sdnhelp
=== SDN Defender Quick Commands ===
wl <mac>            → aggiunge MAC alla whitelist
unwl <mac>          → rimuove MAC dalla whitelist
bl <dpid> <in_port> <src_mac> <dst_mac> → blocca un flusso
unbl <dpid> <in_port> <src_mac> <dst_mac> → sblocca un flusso
list_wl             → mostra la whitelist corrente
list_bl             → mostra la lista dei flussi bloccati
smag01@smag01:~$ wl 00:00:00:00:00:01
[WHITELIST] Adding MAC 00:00:00:00:00:01
{"ok":true}
```

Figura 4.14: Comando `sdnhelp`: elenco delle azioni disponibili nel controller, tra cui l'aggiunta e la rimozione di MAC dalla whitelist.

Per aggiungere l'host `H1` alla whitelist è sufficiente utilizzare l'alias:

```
wl 00:00:00:00:00:01
```

che internamente richiama la stessa chiamata REST già utilizzata in precedenza:

```
curl -s -X POST "$BASE_URL/whitelist" \
  -H "Content-Type: application/json" \
  -d "{\"mac\":\"<mac>\"}"
```

### Comandi di esecuzione

```
mininet> h3 iperf3 -s -p 5201 -D
mininet> h1 iperf3 -c 10.0.0.3 -p 5201 -u -b 50M -t 20 -i
```

L'host `H1`, ora presente nella whitelist, invia un flusso UDP costante di 50 Mbit/s per 20 s. Poiché il suo indirizzo MAC è contrassegnato come trusted, il controller ignora eventuali superamenti di soglia e non applica azioni di `LIMIT` o `BLOCK`.

## 4.5.6 Evidenze sperimentali

Durante l'intero test, il throughput risulta costante e privo di interruzioni, come mostrato nella Figura 4.15. Il log del controller (Figura 4.16) conferma l'accettazione della richiesta REST e l'aggiunta del MAC `00:00:00:00:00:01` alla whitelist.

```
root@smag01:/home/smag01/vmshare/sdn_defender/topology# iperf3 -s -p 5201 &
[1] 3466
root@smag01:/home/smag01/vmshare/sdn_defender/topology# -----------------------
-----------------------------------
Server listening on 5201
-----------------------------------------------------------
Accepted connection from 10.0.0.1, port 45306
[  7] local 10.0.0.3 port 5201 connected to 10.0.0.1 port 45281
[ ID] Interval           Transfer     Bitrate         Jitter    Lost/Total Datag
rams
[  7]   0.00-1.00   sec   612 KBytes  5.01 Mbits/sec  1.731 ms  0/433 (0%)
[  7]   1.00-2.00   sec   711 KBytes  5.83 Mbits/sec  1.795 ms  0/503 (0%)
[  7]   2.00-3.00   sec   713 KBytes  5.84 Mbits/sec  0.799 ms  818/1322 (62%)
[  7]   3.00-4.00   sec   711 KBytes  5.83 Mbits/sec  0.885 ms  2086/2589 (81%)

[  7]   4.00-5.00   sec   711 KBytes  5.83 Mbits/sec  0.844 ms  2086/2589 (81%)

[  7]   5.00-6.00   sec   713 KBytes  5.84 Mbits/sec  0.690 ms  3729/4233 (88%)

[  7]   6.00-7.00   sec   711 KBytes  5.83 Mbits/sec  0.496 ms  3811/4314 (88%)

[  7]   7.00-8.00   sec   711 KBytes  5.83 Mbits/sec  0.710 ms  3812/4315 (88%)

[  7]   8.00-9.00   sec   713 KBytes  5.84 Mbits/sec  0.799 ms  3811/4315 (88%)

[  7]   9.00-10.00  sec   711 KBytes  5.83 Mbits/sec  0.709 ms  3815/4318 (88%)

[  7]  10.00-11.00  sec   711 KBytes  5.83 Mbits/sec  0.688 ms  3802/4305 (88%)

[  7]  11.00-12.00  sec   713 KBytes  5.84 Mbits/sec  0.659 ms  3820/4324 (88%)

[  7]  12.00-13.00  sec   711 KBytes  5.83 Mbits/sec  0.479 ms  3812/4315 (88%)

[  7]  13.00-14.00  sec   711 KBytes  5.83 Mbits/sec  0.772 ms  3817/4320 (88%)

[  7]  14.00-15.00  sec   713 KBytes  5.84 Mbits/sec  0.667 ms  3807/4311 (88%)

[  7]  15.00-16.00  sec   713 KBytes  5.84 Mbits/sec  0.550 ms  3818/4322 (88%)

[  7]  16.00-17.00  sec   711 KBytes  5.83 Mbits/sec  0.609 ms  3807/4310 (88%)

[  7]  17.00-18.00  sec   711 KBytes  5.82 Mbits/sec  0.554 ms  3798/4301 (88%)

[  7]  18.00-19.00  sec   711 KBytes  5.83 Mbits/sec  0.636 ms  3832/4335 (88%)

[  7]  19.00-20.00  sec   711 KBytes  5.83 Mbits/sec  0.782 ms  3812/4315 (88%)

[  7]  20.00-21.00  sec   713 KBytes  5.83 Mbits/sec  0.595 ms  3814/4318 (88%)
```

Figura 4.15: Host `H3`: ricezione continua del traffico UDP proveniente da `H1` dopo l'inserimento in whitelist.

```
(3066) accepted ('127.0.0.1', 43998)
[API ADD] Whitelist MAC 00:00:00:00:00:01
127.0.0.1 - - [28/Oct/2025 13:42:28] "POST /policy/whitelist HTTP/1.1" 200 127 0.001947
```

Figura 4.16: Log del controller Ryu: chiamata REST `POST /policy/whitelist` accettata e aggiunta dell'host `H1` alla whitelist.

L'output del comando restituisce conferma positiva:

```
[WHITELIST] Adding MAC 00:00:00:00:00:01
{"ok":true}
```

indicando che l'operazione è andata a buon fine. Successivamente, è disponibile anche l'azione duale `unwl <mac>`, che consente di rimuovere lo stesso host dalla whitelist.

### 4.5.7 Risultati e validazione

Il test dimostra che l'aggiunta di un MAC alla whitelist esclude l'host dalle politiche di mitigazione, anche in presenza di traffico intenso. Il controller:

- riconosce correttamente le richieste REST di aggiunta alla whitelist;
- evita l'applicazione di azioni di limitazione o blocco sugli host trusted;
- consente la rimozione rapida tramite comando `unwl`.

Il meccanismo di whitelist contribuisce quindi a ridurre i falsi positivi e ad aumentare la stabilità complessiva del sistema, garantendo che gli host affidabili mantengano la connettività anche in condizioni di traffico elevato.

### 4.5.8 Test 3 – Blocklist REST API e blocco manuale

**Scenario di Test.** Il terzo esperimento verifica il corretto funzionamento della **blocklist REST API**, che consente all'amministratore di bloccare o sbloccare manualmente specifici flussi in tempo reale, senza dover modificare le tabelle di flusso o riavviare il controller. Questo meccanismo rappresenta un ulteriore livello di controllo amministrativo, complementare alla mitigazione automatica implementata dal modulo `PolicyEngine`.

**Comandi di esecuzione:**

```
mininet> h3 iperf3 -s -p 5201 -D
mininet> h1 iperf3 -c 10.0.0.3 -p 5201 -u -b 2M -t 60 -i
```

L'host `H1` invia un flusso UDP costante di 2 Mbit/s verso `H3`. Durante la trasmissione, il flusso viene prima bloccato e poi sbloccato manualmente tramite le API REST, simulate attraverso gli alias Bash `bl` e `unbl`.

I due comandi sono stati aggiunti al file `.bashrc` insieme agli altri alias del modulo `sdnhelp`. Eseguendo il comando:

```
sdnhelp
```

si ottiene l'elenco completo delle operazioni disponibili, come illustrato in Figura 4.17.

```
smag01@smag01:~$ sdnhelp
=== SDN Defender Quick Commands ===
wl <mac>            → aggiunge MAC alla whitelist
unwl <mac>          → rimuove MAC dalla whitelist
bl <dpid> <in_port> <src_mac> <dst_mac> → blocca un flusso
unbl <dpid> <in_port> <src_mac> <dst_mac> → sblocca un flusso
list_wl             → mostra la whitelist corrente
list_bl             → mostra la lista dei flussi bloccati
smag01@smag01:~$ bl 1 1 00:00:00:00:00:01 00:00:00:00:00:03
[BLOCK] Flow 00:00:00:00:00:01 → 00:00:00:00:00:03 on dpid=1
{"ok":true}
smag01@smag01:~$ unbl 1 1 00:00:00:00:00:01 00:00:00:00:00:03
[UNBLOCK] Flow 00:00:00:00:00:01 → 00:00:00:00:00:03 on dpid=1
{"ok":true}
```

Figura 4.17: Comando `sdnhelp`: elenco delle azioni disponibili, tra cui i comandi `bl` e `unbl` per bloccare e sbloccare manualmente i flussi.

Gli alias Bash implementano internamente le seguenti chiamate REST:

```
# Blocco manuale di un flusso
curl -s -X POST "$BASE_URL/blockflows" \
  -H "Content-Type: application/json" \
  -d "{\"dpid\":<id>,\"in_port\":<port>,\"eth_src\":\"<sr


# Sblocco manuale dello stesso flusso
curl -s -X DELETE "$BASE_URL/blockflows" \
  -H "Content-Type: application/json" \
  -d "{\"dpid\":<id>,\"in_port\":<port>,\"eth_src\":\"<sr
```

### 4.5.9 Evidenze sperimentali

La Figura 4.18 mostra l'output del server `H3`: inizialmente il traffico di `H1` viene ricevuto normalmente, poi interrotto dopo il blocco manuale e infine ripristinato a seguito dell'operazione di sblocco.

```
"Node: h3"
root@smag01:/home/smag01/vmshare/sdn_defender/topology# iperf3 -s -p 5201 &
[1] 4045
root@smag01:/home/smag01/vmshare/sdn_defender/topology# -----------------------
-----------------------------------
Server listening on 5201
-----------------------------------------------------------
Accepted connection from 10.0.0.1, port 55548
[  7] local 10.0.0.3 port 5201 connected to 10.0.0.1 port 45636
[ ID] Interval           Transfer     Bitrate         Jitter    Lost/Total Datag
rams
[  7]   0.00-1.00   sec   211 KBytes  1.73 Mbits/sec  0.353 ms  0/149 (0%)
[  7]   1.00-2.00   sec   245 KBytes  2.00 Mbits/sec  0.420 ms  0/173 (0%)
[  7]   2.00-3.00   sec   219 KBytes  1.79 Mbits/sec  0.660 ms  0/155 (0%)
[  7]   3.00-4.00   sec  0.00 Bytes  0.00 bits/sec  0.660 ms  0/0 (0%)
[  7]   4.00-5.00   sec  0.00 Bytes  0.00 bits/sec  0.660 ms  0/0 (0%)
[  7]   5.00-6.00   sec  0.00 Bytes  0.00 bits/sec  0.660 ms  0/0 (0%)
[  7]   6.00-7.00   sec  0.00 Bytes  0.00 bits/sec  0.660 ms  0/0 (0%)
[  7]   7.00-8.00   sec  0.00 Bytes  0.00 bits/sec  0.660 ms  0/0 (0%)
[  7]   8.00-9.00   sec  0.00 Bytes  0.00 bits/sec  0.660 ms  0/0 (0%)
[  7]   9.00-10.00  sec  0.00 Bytes  0.00 bits/sec  0.660 ms  0/0 (0%)
[  7]  10.00-11.00  sec  0.00 Bytes  0.00 bits/sec  0.660 ms  0/0 (0%)
[  7]  11.00-12.00  sec  0.00 Bytes  0.00 bits/sec  0.660 ms  0/0 (0%)
[  7]  12.00-13.00  sec  0.00 Bytes  0.00 bits/sec  0.660 ms  0/0 (0%)
[  7]  13.00-14.00  sec   209 KBytes  1.72 Mbits/sec  0.372 ms  1769/1917 (92%)

[  7]  14.00-15.00  sec   243 KBytes  1.99 Mbits/sec  0.470 ms  0/172 (0%)
[  7]  15.00-16.00  sec   225 KBytes  1.84 Mbits/sec  0.509 ms  14/173 (8.1%)
[  7]  16.00-17.00  sec   117 KBytes   962 Kbits/sec  0.566 ms  88/171 (51%)
[  7]  17.00-18.00  sec   119 KBytes   973 Kbits/sec  0.395 ms  89/173 (51%)
[  7]  18.00-19.00  sec   119 KBytes   973 Kbits/sec  0.567 ms  89/173 (51%)
```

Figura 4.18: Host `H3`: ricezione regolare, interruzione del flusso durante il blocco manuale e successiva ripresa dopo lo sblocco.

Nel log del controller (Figure 4.19 e 4.20) si osservano le richieste REST `POST` e `DELETE` inviate all'endpoint `/policy/blockflows`, rispettivamente associate alle azioni `Installing DROP (flow)` e `Removing DROP (flow)`.

```
(3787) accepted ('127.0.0.1', 56890)
[API ADD] Block flow 00:00:00:00:00:01 → 00:00:00:00:00:03 (dpid=1, in_port=1)
Installing DROP (flow) FlowKey(dpid=1, in_port=1, eth_src='00:00:00:00:00:01', eth_dst=
'00:00:00:00:00:03', ip_src=None, ip_dst=None, tp_dst=None)
127.0.0.1 - - [28/Oct/2025 14:07:25] "POST /policy/blockflows HTTP/1.1" 200 127 0.00570
3
```

Figura 4.19: Log del controller Ryu: richiesta `POST /policy/blockflows` accettata e installazione della regola di `DROP`.

```
(3787) accepted ('127.0.0.1', 37112)
[API DEL] Unblock flow 00:00:00:00:00:01 → 00:00:00:00:00:03 (dpid=1, in_port=1)
Removing DROP (flow) FlowKey(dpid=1, in_port=1, eth_src='00:00:00:00:00:01', eth_dst='0
0:00:00:00:00:03', ip_src=None, ip_dst=None, tp_dst=None)
127.0.0.1 - - [28/Oct/2025 14:07:36] "DELETE /policy/blockflows HTTP/1.1" 200 127 0.003
049
```

Figura 4.20: Log del controller Ryu: richiesta `DELETE /policy/blockflows` accettata e rimozione della regola di `DROP`.

L'output dei comandi sul terminale conferma la corretta esecuzione:

```
[BLOCK] Flow 00:00:00:00:00:01 → 00:00:00:00:00:03 on dp
{"ok":true}
[UNBLOCK] Flow 00:00:00:00:00:01 → 00:00:00:00:00:03 on
{"ok":true}
```

### 4.5.10 Risultati e validazione

Il test conferma il corretto funzionamento della blocklist manuale via REST API:

- il controller riceve e processa correttamente le chiamate di blocco e sblocco;
- le regole `DROP` vengono installate e rimosse in tempo reale;
- la connettività dell'host vittima viene interrotta solo durante il blocco manuale;
- il sistema risponde con messaggi di conferma JSON coerenti.

L'integrazione degli alias `bl` e `unbl` nel file `.bashrc` rende il processo di blocco e sblocco intuitivo e immediato per l'amministratore, con-

sentendo di agire in modo selettivo sui flussi sospetti senza interferire con gli altri componenti del sistema.

# Capitolo 5

# Conclusioni

Il lavoro svolto ha avuto come obiettivo la progettazione e lo sviluppo di un **firewall SDN basato su Ryu** capace di reagire in modo adattivo e proporzionale agli attacchi di tipo DoS e DDoS. L'attività ha rappresentato un'evoluzione significativa rispetto all'implementazione di partenza, superandone i limiti di rigidità, scarsa modularità e over-blocking.

L'analisi iniziale ha portato all'individuazione di cinque principali criticità nella versione originale, affrontate attraverso un approccio incrementale e una riorganizzazione architetturale che ha portato alla nascita di una piattaforma più chiara, estendibile e conforme ai principi di progettazione moderna dei sistemi SDN.

Le principali innovazioni introdotte sono:

- **Architettura modulare e disaccoppiata:** il controller è stato suddiviso in moduli indipendenti — `monitor.py`, `policy.py`,

`enforcement.py`, `datastore.py`, `config.py` e `models.py` — coordinati dal file `controller_app.py`. Ogni componente svolge un ruolo preciso: il `Monitor` raccoglie statistiche in tempo reale, il `PolicyEngine` analizza le soglie e decide le azioni di difesa, l'`Enforcer` le applica sugli switch tramite OpenFlow, e il `DataStore` mantiene lo stato condiviso e le soglie dinamiche.

- **Soglie dinamiche adattive (EWMA):** la detection non si basa più su valori statici ma su soglie aggiornate dinamicamente, calcolate tramite media mobile esponenziale (`update_dynamic_flow_th` in `datastore.py`). Questo ha risolto il problema della "Static Threshold", permettendo al controller di adattarsi al traffico reale e ridurre drasticamente i falsi positivi.

- **Mitigazione progressiva:** nel modulo `enforcement.py` sono stati introdotti due livelli di reazione:

  - `LIMIT` — applica un rate limiting a 1 Mbps tramite `Meter Table`, riducendo la banda dei flussi borderline senza interrompere la connessione;
  - `BLACKHOLE` — installa una regola di `DROP` temporaneo multi-switch (20 s di default) in caso di DDoS aggregato, rimuovendo automaticamente la regola dopo la durata configurata.

Questo approccio multilivello, gestito in modo coordinato dal `PolicyEngine` e dall'`Enforcer`, consente una difesa più flessibile e proporzionale.

- **Metriche di detection avanzate:** il modulo `policy.py` combina varianza, coefficiente di variazione e rapporto picco/EWMA per riconoscere anche pattern di attacco *burst* e *stealth*. Tali metriche vengono valutate per ciascun flusso grazie alla struttura dati `FlowKey`, rendendo l'analisi granulare e precisa.

- **Politiche flessibili di blocking/unblocking:** nel modulo `datastore.p` sono state aggiunte **whitelist MAC** e **blocklist REST** gestibili dinamicamente, mentre il `PolicyEngine` integra un **cooldown temporale** prima dello sblocco (`unblock_cooldown_s`), prevenendo cicli rapidi di blocco/sblocco.

La fase di **testing e validazione** ha confermato la piena efficacia delle soluzioni:

- il problema dell'over-blocking è stato risolto, poiché le azioni vengono applicate a livello di flusso e non di porta;

- i meccanismi di `LIMIT` e `BLOCK` operano in modo coordinato, garantendo continuità di servizio ai flussi legittimi;

- il sistema è in grado di rilevare e mitigare attacchi *burst* e distribuiti, mantenendo la stabilità delle soglie nel tempo;

- le regole di mitigazione sono dinamiche e reversibili, con logging dettagliato delle azioni eseguite.

Parallelamente, è stata progettata una **topologia estesa** (`topology_wide.py`) con nove switch e sei host, che consentirà di valutare la scalabilità e la coerenza delle policy in reti multi-hop. Tale topologia è stata completamente integrata nel progetto, pur non essendo ancora sottoposta a validazione sperimentale.

In conclusione, il lavoro ha trasformato un prototipo accademico in una piattaforma di difesa SDN più completa, robusta e realistica. Grazie alla nuova architettura modulare, alle soglie adattive e alle azioni di mitigazione multilivello, il controller SDN Defender è ora in grado di distinguere in modo accurato tra traffico legittimo e malevolo, adottando strategie di difesa proporzionate e confermando l'efficacia del paradigma **Software Defined Networking** come base per le architetture di sicurezza di nuova generazione.